

# Salesforce Concepts You Need to Master

A practical reference mapped to the Bhartiya City Real Estate CRM implementation

**What this document is.** The Bhartiya City scope document (1,674 estimated hours, 19 BRN areas, Lead through Possession and Resale) touches nearly every capability in the Salesforce platform. This reference walks through each of those capabilities, explains what it is and when to use it, and then maps it directly to the specific BRN line items where it shows up. Where a capability requires a named skill (SAP integration, CTI, document generation), the skills needed are listed explicitly.

**How to read it.** Each section follows the same shape: the concept, the syntax or configuration pattern, common mistakes, and an orange *In this project* callout showing exactly which BRN rows depend on it. Skim the callouts first if you want to know what to learn in priority order; read the concept sections when you need depth.

**Scope note.** This is a platform-concepts reference, not a design document. It does not decide the Bhartiya City solution - it tells you what you need to be fluent in to build, review or defend that solution.

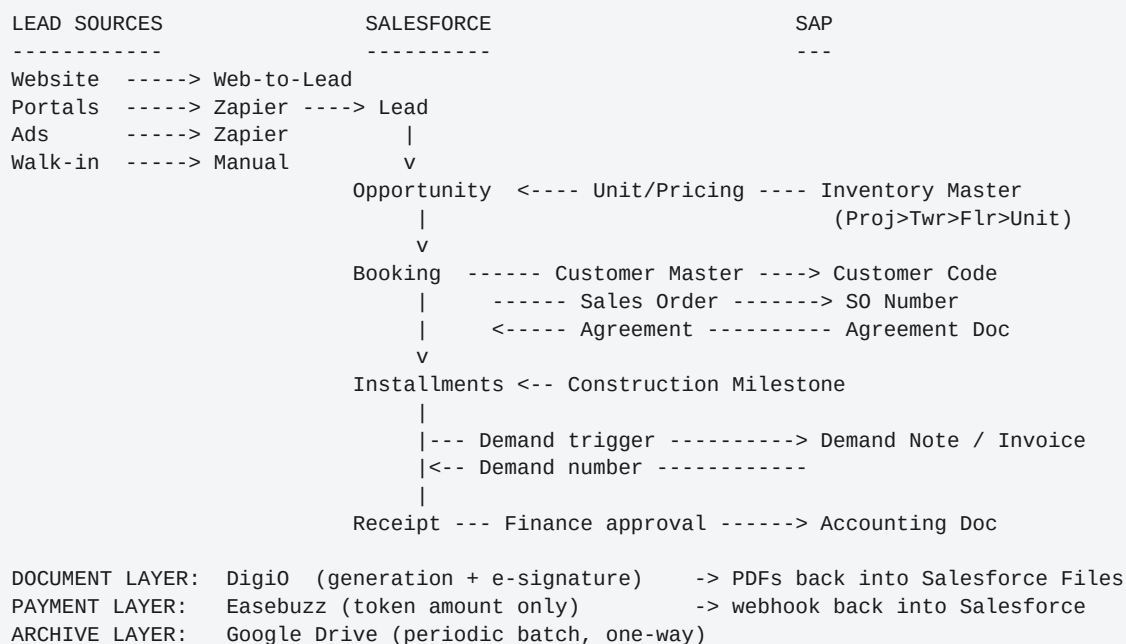
| Part                          | Covers                                                               | Priority  |
|-------------------------------|----------------------------------------------------------------------|-----------|
| 1. Data model foundations     | Objects, relationships, Person Accounts, record types, External IDs  | Critical  |
| 2. Declarative automation     | Validation rules, Flow types, order of execution, approvals          | Critical  |
| 3. Apex                       | Triggers, handler pattern, async Apex, governor limits, testing      | Critical  |
| 4. LWC                        | Components, wire, imperative Apex, events, Screen Flow embedding     | High      |
| 5. Integration core           | REST callouts, Named Credentials, inbound Apex REST, Platform Events | Critical  |
| 6. Named integrations         | SAP, DigiO, Easebuzz, CTI/Open CTI, Zapier, Google Drive             | Critical  |
| 7. Security and sharing       | OWD, sharing rules, Apex managed sharing, FLS                        | High      |
| 8. Documents and files        | ContentVersion, document generation patterns, e-signature            | High      |
| 9. Public-facing capture      | Experience Cloud, Salesforce Sites, Web-to-Lead, OTP flows           | Medium    |
| 10. Reporting and data        | Report types, dashboards, Bulk API, migration strategy               | Medium    |
| 11. Skills matrix and roadmap | What to learn, in what order, with checkpoints                       | Reference |

# 0. Orientation: the architecture you are building inside

Before the platform concepts, fix the shape of the solution in your head. Almost every design decision in this project falls out of three structural facts.

## 0.1 Two systems of record, split by concern

SAP is the ERP and owns financial and inventory truth: Customer Master, Sales Orders, Agreements, Demand Notes, unit pricing, unit status, construction milestones. Salesforce is the system of engagement and owns process and relationship: who is working the lead, what stage the deal is at, which documents have moved, which approvals have cleared.



**Why this matters for your learning:** roughly a third of the total effort in this scope is integration or integration-adjacent. If you are strong on Flow and Apex but weak on callouts, Named Credentials, Platform Events and error handling, you will be blocked on the highest-value rows in the sheet.

## 0.2 Declarative where possible, Apex where the platform runs out

The scope sheet classifies every row as Config, Custom, or a blend. That classification is not arbitrary - it reflects specific platform ceilings. Learn to recognise the ceilings, because in a review someone will ask you to justify why a row is Custom.

| Ceiling hit                  | Why declarative fails                                                                                                                                                    | Example row                    |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| External callout needed      | Flow cannot make an authenticated HTTP callout to an arbitrary REST endpoint with custom headers, retry and logging. Needs Apex (or External Services for simple cases). | SAP sync rows, DigiO, Easebuzz |
| Complex ranking or matching  | Flow can filter records but ranking by weighted criteria across many attributes with configurable rules needs Apex.                                                      | BRN-04.3 Property Matching     |
| Row-level sharing at runtime | Sharing rules are criteria or owner based. Granting access to a specific record to a specific user at a specific moment needs Apex Managed Sharing.                      | BRN-05.2 Booking Handover      |
| High-volume scheduled math   | Interest on overdue installments across every booking exceeds Flow element limits; needs Batch Apex.                                                                     | BRN-06.9 Overdue Interest      |

| Ceiling hit                                        | Why declarative fails                                                                                       | Example row                         |
|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------|
| Cross-object numeric rollups without master-detail | Roll-up summary fields require master-detail. Lookup relationships need Flow or Apex rollup logic.          | Site visit counts, open snag counts |
| Bulk data reshaping                                | Migration of historical data with cleansing, dedup and relationship linking needs Bulk API plus Batch Apex. | BRN-19.5 Historical Migration       |

### 0.3 Everything is auditable

Look at how often the scope says 'maintain audit history', 'capture approver, decision date/time and comments', 'maintain integration status and request/response reference'. In a regulated, money-handling, legally-binding-document business, audit is not a nice to have. Practically this means you must be fluent in Field History Tracking, in designing log objects, and in never silently swallowing an exception.

# 1. Data Model Foundations

## 1.1 Standard vs custom objects - and when to choose which

Standard objects come with free platform behaviour you would otherwise rebuild: Lead conversion, Opportunity stage and forecasting, Campaign influence, Activity timeline, duplicate management, and a large library of standard report types. Custom objects give you a clean schema but you build everything yourself.

The default heuristic: **use the standard object when the standard lifecycle matches the business lifecycle.** Deviate only when the mismatch is structural, not cosmetic.

| Business concept               | Object choice in this scope      | Reasoning                                                                                              |
|--------------------------------|----------------------------------|--------------------------------------------------------------------------------------------------------|
| Enquiry                        | Standard Lead                    | Needs conversion, dedup, Web-to-Lead, assignment rules - all standard.                                 |
| Buyer                          | Person Account                   | Buyer is an individual, but needs Bookings, Payments, Documents rolling up like an Account.            |
| Channel Partner firm           | Business Account + Contacts      | Firm is a company; individual brokers are people under it.                                             |
| Deal in progress               | Standard Opportunity             | Stage progression, Closed Won/Lost, line items for tagged units.                                       |
| Site Visit                     | Custom object (or Task)          | Needs its own lifecycle, reschedule, multiple per Opportunity, rollup counts.                          |
| Project / Tower / Floor / Unit | Custom objects                   | No standard equivalent for a four-level physical inventory hierarchy.                                  |
| Booking                        | Custom object                    | Post-sale lifecycle differs completely from Opportunity; needs separate ownership.                     |
| Payment Plan / Milestone       | Custom master-detail pair        | Reusable template master data with child milestone rows.                                               |
| Installment / Demand / Receipt | Custom objects                   | Financial transaction records mirroring SAP documents.                                                 |
| Ticket                         | Case (standard) or custom Ticket | Case gives you Entitlements, Web-to-Case, Email-to-Case and future Experience Cloud exposure for free. |

**Interview-grade point:** the scope hedges on Ticket - 'Custom Ticket object (or Case object)'. The right answer is almost always Case, because BRN-11.1 explicitly says the module must extend to an Experience Cloud customer portal later. Case has native portal support, Entitlement/Milestone SLA tracking, and Omni-Channel routing. Rebuilding those on a custom object is wasted effort.

## 1.2 Relationships: lookup vs master-detail vs hierarchical

| Type          | Behaviour                                                                                                                                                | Use when                                                                                     |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Lookup        | Loose link. Child survives parent deletion (configurable). Independent sharing. No rollup summary fields. Up to 40 per object.                           | Booking -> Sales Executive; Lead -> Channel Partner; Opportunity -> Payment Plan.            |
| Master-Detail | Tight link. Child deleted with parent. Child inherits parent sharing and has no owner. Enables roll-up summary fields. Max 2 per object. Required field. | Payment Plan -> Payment Milestone; Booking -> Installment (if cascade delete is acceptable). |
| Hierarchical  | Special self-lookup available only on User.                                                                                                              | Manager field for approval routing.                                                          |

| Type                              | Behaviour                                                                                                                             | Use when                                                                |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| External Lookup / Indirect Lookup | Links to an External Object surfaced via Salesforce Connect (OData). Indirect lookup matches on an External ID rather than record Id. | Only relevant if SAP data is ever surfaced live rather than replicated. |

## The junction object pattern

A junction object is a custom object with two master-detail relationships, used to model a many-to-many relationship. The classic example is a student who can enrol in many courses and a course that has many students - Enrolment is the junction.

```
Booking__c <---- (Master-Detail) ---- Booking_Applicant__c ---- (Master-Detail/Lookup) ----> Contact
|
+-- Applicant_Role__c (Primary / Co-Applciant / Family
)
+-- Relationship__c
+-- Is_Financial_Applicant__c
+-- KYC_Status__c
```

**In this project:** BRN-05.3 explicitly calls for this: *'Booking Applicants to be created as a junction object between Contact and Booking objects.'* One person can appear on several bookings (an investor, or a family member co-applying on two flats), and one booking has several applicants. Without the junction you cannot support BRN-05.14 Clash Case Identification, which depends on finding the same person across multiple bookings.

## 1.3 Person Accounts - the irreversible decision

A Person Account is a single record that Salesforce stores internally as an Account plus a Contact, presented as one record. It exists because some businesses sell to individuals, not companies, but still need everything that hangs off an Account - Opportunities, Cases, related records, rollups.

### What you must know

- Enabling Person Accounts is **irreversible** and requires a Salesforce Support case. It changes Account behaviour org-wide.
- Once enabled, the Account object has record types for both Business Accounts and Person Accounts. `IsPersonAccount` is a formula-accessible boolean.
- Person Account fields are stored with a PersonX API naming convention - `PersonEmail`, `PersonMobilePhone`, `PersonContactId`.
- Some report types, list views and managed packages behave differently or need rework.
- You cannot have a Person Account as the parent in a Contact-to-Account relationship in the usual way, which occasionally surprises people building household models.

```
// Querying across both types safely
List<Account> buyers = [
    SELECT Id, Name, PersonEmail, PersonMobilePhone, PersonContactId
    FROM Account
    WHERE IsPersonAccount = true
    AND Id IN :bookingAccountIds
];

// A Business Account (Channel Partner firm) with its broker Contacts
List<Account> firms = [
    SELECT Id, Name, RERA_Registration__c,
        (SELECT Id, Name, Email FROM Contacts)
    FROM Account
    WHERE RecordType.DeveloperName = 'Channel_Partner'
];
```

**In this project:** BRN-02.1. The scope flags this correctly as needing to be 'confirmed and sequenced early in the project'. If you are giving inputs back on this document, this is a genuine go/no-go decision point, not a technical

footnote - it should be an explicit client sign-off item before any build starts, because unwinding it later means rebuilding the org.

## 1.4 Record Types

Record Types let one object serve several business processes: different page layouts, different picklist value subsets, and different assigned Sales Processes or Support Processes, all governed by profile or permission set assignment.

| Use case in scope                 | Object         | Why a record type                                                                                                                                  |
|-----------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Resale vs fresh sale              | Opportunity    | BRN-15.3 - resale reuses the same lifecycle but bypasses some fresh-sale validations and shows extra fields (original booking, previous customer). |
| Customer vs Channel Partner       | Account        | BRN-02.1 - Person Account record type for buyers, Business Account record type for broker firms.                                                   |
| Sales Query / CRM Query / General | Case or Ticket | BRN-11.1 - different routing, different layouts, different SLA.                                                                                    |

### Record type vs conditional visibility - a decision you will be asked to defend

BRN-01.1 deliberately chooses *conditional field visibility on one layout* rather than record types for Quick Entry vs Walk-in leads. That is the right call: record types carry permanent overhead (profile assignment, layout maintenance, reporting complexity) and should be reserved for genuinely different *processes*, not for showing or hiding a couple of fields. Dynamic Forms and component visibility rules handle the latter far more cheaply.

## 1.5 External IDs and idempotency

An External ID is a custom field marked as an external unique identifier. Two capabilities make it essential in any integration project:

- **Upsert by External ID** - you can insert-or-update a record without first querying for its Salesforce Id, using a key that the external system already knows.
- **Idempotency** - if the same message arrives twice (a retried webhook, a replayed Zap, a duplicated SAP push), upserting on External ID updates the existing record instead of creating a duplicate.

```
// Upsert a Unit using the SAP material code as the key
Unit__c u = new Unit__c(
    SAP_Material_Code__c = payload.materialCode,    // External Id, Unique
    Status__c            = payload.status,
    Base_Price__c        = payload.basePrice
);
upsert u SAP_Material_Code__c;

// Bulk version - the pattern you will actually write
List<Unit__c> units = buildUnitsFromPayload(payloadList);
Database.UpsertResult[] results =
    Database.upsert(units, Unit__c.SAP_Material_Code__c, false); // allOrNone = false

for (Integer i = 0; i < results.size(); i++) {
    if (!results[i].isSuccess()) {
        logIntegrationError('SAP_UNIT_SYNC', units[i].SAP_Material_Code__c,
            results[i].getErrors()[0].getMessage());
    }
}
```

**In this project:** External IDs appear all over this scope: portal enquiry IDs from 99acres/MagicBricks (BRN-01.3), Facebook/Google form submission IDs (BRN-01.4), SAP Customer Code (BRN-05.5), SAP Sales Order Number (BRN-05.7), SAP material/unit code (BRN-09.1), Demand Note number (BRN-06.3), Easebuzz transaction ID (BRN-18.4), DigiO document ID (BRN-18.3). If you take one thing from this section: **every integration boundary needs an External ID on the Salesforce side.**

## 1.6 Custom Metadata Types vs Custom Settings

Both hold configuration rather than transactional data. The difference matters because the scope leans on configurable thresholds repeatedly.

|                         | Custom Metadata Types                                                       | Custom Settings (Hierarchy)                                     |
|-------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------|
| Deployable              | Yes - records deploy with metadata through change sets and packages.        | No - the definition deploys, the data does not.                 |
| Queryable in Apex       | SOQL, and does not count against SOQL query limits.                         | Retrieved via getInstance(), cached.                            |
| Per-user/profile values | No                                                                          | Yes - hierarchy of org, profile, user.                          |
| Testable                | Visible in tests without SeeAllData.                                        | Needs to be inserted in test setup.                             |
| Best for                | SLA thresholds, ladder slabs, field mappings, template IDs, routing tables. | Feature toggles, per-user overrides, integration on/off switch. |

**In this project:** The scope specifies Custom Metadata for: SLA thresholds of 24 hours and 60 days (BRN-01.12), fallback project/team routing (BRN-01.6), Digio Template IDs and field mappings (BRN-18.3, and every document row), ideal TAT per stage (BRN-04.12, BRN-05.13), cancellation ladder slabs (BRN-08.4), archival frequency (BRN-18.2), Easebuzz merchant config (BRN-18.4). Learn to build a metadata-driven design - it is the single most repeated architectural instruction in the whole document.

### Worked example: the cancellation ladder as metadata

```
// Cancellation_Ladder__mdt (master)
// Effective_From__c, Effective_To__c, Project__c, Active__c
// Cancellation_Ladder_Slab__mdt (child rows, or a single mdt with From/To)
// From_Day__c, To_Day__c, Charge_Percent__c, Fixed_Charge__c

public with sharing class CancellationChargeCalculator {

    public static Decimal calculate(Decimal bookingValue, Integer elapsedDays, Id projectId) {
        for (Cancellation_Ladder_Slab__mdt slab : [
            SELECT From_Day__c, To_Day__c, Charge_Percent__c, Fixed_Charge__c
            FROM Cancellation_Ladder_Slab__mdt
            WHERE Active__c = true
            ORDER BY From_Day__c ASC]) {

            if (elapsedDays >= slab.From_Day__c && elapsedDays <= slab.To_Day__c) {
                return slab.Fixed_Charge__c != null
                    ? slab.Fixed_Charge__c
                    : bookingValue * (slab.Charge_Percent__c / 100);
            }
        }
        return 0; // no slab matched - surface this, do not silently return zero
    }
}
```

**Common mistake:** hardcoding the slabs in Apex because 'the business will confirm them later'. The scope explicitly says the slabs are unconfirmed and that this row 'covers the configurable framework only'. Build the framework; let the numbers arrive as data.

## 2. Declarative Automation

### 2.1 The order of execution - memorise this

Every save in Salesforce runs a fixed sequence. Almost every 'why did my automation not work' bug traces back to not knowing this order. You will be asked it in interviews and you will need it to debug this project.

1. Record loaded from database (or initialised for insert)
2. New field values overwrite old values
3. System validation (required fields, field types, max length)
4. BEFORE-SAVE record-triggered FLOWS <-- fast, no DML needed
5. BEFORE triggers (before insert / before update)
6. System validation again + custom validation rules
7. Duplicate rules
8. Record saved to database (NOT committed)
9. AFTER triggers (after insert / after update)
10. Assignment rules (Lead / Case)
11. Auto-response rules
12. Workflow rules (legacy)
13. Processes (legacy PB) and AFTER-SAVE record-triggered FLOWS
14. Escalation rules
15. Roll-up summary fields recalculated on parent -> parent may re-run steps 3-15
16. Criteria-based sharing evaluation
17. COMMIT -> post-commit logic: emails, async paths, Platform Event publish

#### The three consequences you must internalise

- **Before-save updates are free.** Setting a field on the record being saved in a before-save Flow or before trigger needs no DML and no extra query. Doing the same thing after save costs a full extra save cycle - roughly ten times slower.
- **Validation runs after before-logic.** If a before-save Flow sets a field to something a validation rule rejects, the save fails - and users blame the validation rule.
- **Roll-up recalculation can cascade.** A child update rolls up to the parent, which re-enters the order of execution on the parent, which may fire the parent's triggers. This is how recursion bugs are born.

**In this project:** Directly relevant to BRN-01.5 (Lead Deduplication). The scope says to use a before-save flow or Apex to normalise phone numbers *before* comparison. That works only because before-save logic (step 4/5) runs before duplicate rules (step 7). Get the order wrong and dedup compares raw, unnormalised numbers and misses every duplicate that arrived with a +91 prefix.

### 2.2 Validation rules

A validation rule is a formula that must evaluate to FALSE for the save to succeed. If it returns TRUE, the save is blocked and the error message shown.

```
/* Lost Reason mandatory only when status = Lost (BRN-01.10) */
AND(
    ISPICKVAL(Status, "Lost"),
    ISBLANK(TEXT(Lost_Reason__c))
)

/* Block Closed Won until every booking prerequisite is met (BRN-04.14) */
AND(
    ISPICKVAL(StageName, "Closed Won"),
    OR(
        NOT(Booking_Form_Approved__c),
        NOT(Token_Payment_Received__c),
        ISBLANK(Blocked_Unit__c)
    )
)

/* Prevent editing a closed Opportunity unless the user has an override permission */
AND(
    ISPICKVAL(PRIORVALUE(StageName), "Closed Won"),
```

```
NOT($Permission.Can_Edit_Closed_Opportunity)
)
```

## Functions worth knowing cold

| Function             | Purpose                                                              |
|----------------------|----------------------------------------------------------------------|
| ISCHANGED(field)     | True when the field value differs from its prior value. Update only. |
| PRIORVALUE(field)    | The value before this save - the basis of state-locking rules.       |
| ISNEW()              | True on insert. Combine with ISCHANGED to scope rules.               |
| ISPICKVAL(f, 'val')  | Compare picklists. Use TEXT(f) for multi-value comparison.           |
| \$Permission.X       | Reference a custom permission - the clean way to build overrides.    |
| \$Profile.Name       | Works, but brittle. Prefer custom permissions.                       |
| REGEX(text, pattern) | Format validation - Indian mobile, PAN, Aadhaar, GSTIN.              |

```
/* Indian mobile number format */
NOT(REGEX(Phone, "^[6-9][0-9]{9}$"))

/* PAN format: five letters, four digits, one letter */
AND(
  NOT(ISBLANK(PAN__c)),
  NOT(REGEX(PAN__c, "[A-Z]{5}[0-9]{4}[A-Z]{1}"))
)
```

**Governor note:** the scope caps validation rules per row (5 here, 15 there). That cap exists because validation rules are evaluated on every save and are hard to debug in bulk. When a requirement needs more than a handful of conditional-mandatory fields, consider Dynamic Forms conditional requiredness or a Flow-driven checklist instead of stacking twenty validation rules on one object.

## 2.3 Flow - the types and when to use each

| Flow type                      | Triggered by                                       | Use for                                                                                                                                      |
|--------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Record-Triggered (before save) | Insert/update, runs at step 4                      | Setting fields on the <i>same</i> record. Normalising phone numbers, stamping dates, calculating formula-like values that need to be stored. |
| Record-Triggered (after save)  | Insert/update, runs at step 13                     | Creating or updating <i>other</i> records, sending notifications, calling Apex, publishing Platform Events.                                  |
| Record-Triggered (delete)      | Before delete                                      | Cleanup, blocking deletion with an error.                                                                                                    |
| Scheduled-Triggered            | A schedule plus optional filter                    | Nightly evaluation across a record set. Lead aging, SLA breach, overdue detection.                                                           |
| Screen Flow                    | User clicks an action or opens a page              | Guided multi-step data entry with branching. Manual assignment, receipt creation, inventory shift initiation.                                |
| Autolaunched (no trigger)      | Called from Apex, another Flow, a process, or REST | Reusable subflow. Shared logic invoked from several places.                                                                                  |
| Platform Event-Triggered       | A Platform Event is published                      | Decoupled integration handling. Reacting to an inbound SAP event.                                                                            |

### Scheduled path vs Scheduled-Triggered Flow

A **scheduled path** lives inside a record-triggered Flow and runs relative to a field on that record ('3 days after Installment Due Date'). A **Scheduled-Triggered Flow** runs on a clock across a filtered set of records ('every night at 1am, all Leads where Last\_Action\_Date < 24 hours ago'). Both appear in this project.

**In this project:** Scheduled paths: site visit reminders (BRN-03.3), payment reminders on demand letters (BRN-06.4), unit hold auto-release on expiry (BRN-04.12). Scheduled-Triggered Flows: 24-hour lead SLA breach and 60-day sales window reversion (BRN-01.12). Batch Apex instead of scheduled flow: overdue interest calculation (BRN-06.9) - because it touches every unpaid installment across every booking and will blow Flow element limits.

## Flow bulkification - the mistake that will break this project

Flow bulkifies automatically *between* elements, but only if you do not put DML or SOQL inside a loop. A Get Records or Update Records element placed inside a Loop element executes once per iteration, which means 200 queries on a 200-record batch.

|                           |                                                                          |
|---------------------------|--------------------------------------------------------------------------|
| <b>BAD</b>                | <b>GOOD</b>                                                              |
| ---                       | ----                                                                     |
| Loop over Bookings        | Get Records: all related Installments<br>(filter: Booking IN collection) |
| Get Records: Installments | Loop over Installments                                                   |
| Assign field              | Assign field                                                             |
| Update Records            | Add to collection variable                                               |
| End Loop                  | End Loop                                                                 |
| => 200 queries, 200 DML   | Update Records: the collection                                           |
|                           | => 1 query, 1 DML                                                        |

## Fault paths

Every element that can fail - Get, Create, Update, Delete, Apex Action, Callout - should have a fault path. Without one, the user sees an unhandled Flow exception email and a stack trace, and the record is left in an inconsistent state.

- Connect the fault path to a Screen element (for Screen Flows) showing a readable message.
- For background flows, write to a custom Integration\_Log\_\_c or Error\_Log\_\_c record.
- Use the \$Flow.FaultMessage global variable to capture the technical detail.
- Notify an admin via email alert or Custom Notification for integration-critical failures.

## 2.4 Approval Processes

A standard Approval Process handles submission, routing, approve/reject/recall, field updates on each outcome, and locks the record while pending. It gives you an audit trail (ProcessInstance and ProcessInstanceStep) for free - which is why the scope reaches for it repeatedly.

### Anatomy

| Component                  | What it does                                                                                                                   |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Entry criteria             | Which records can be submitted at all.                                                                                         |
| Initial submission actions | Field updates, email alerts, tasks fired on submission. The record lock happens here.                                          |
| Approval steps (1..n)      | Each step has criteria, an approver (manager, queue, related user, specific user), and rules for delegation/parallel approval. |
| Final approval actions     | Field updates and automation on approval - e.g. set Status = Approved, then unlock or keep locked.                             |
| Final rejection actions    | Set status, notify submitter, unlock for correction.                                                                           |
| Recall actions             | What happens if the submitter pulls the request back.                                                                          |

### Approval processes in this project

| BRN  | Approval                     | Levels | Notes                                  |
|------|------------------------------|--------|----------------------------------------|
| 04.9 | Booking Form approval by CRM | 1      | Triggers Booking creation on approval. |

| BRN  | Approval                                      | Levels            | Notes                                                                                    |
|------|-----------------------------------------------|-------------------|------------------------------------------------------------------------------------------|
| 05.4 | Booking-level gate before SAP Customer Master | 2 max             | Blocks the SAP sync until approved.                                                      |
| 06.6 | Receipt approval by Finance                   | 2 max             | Finance cross-checks in SAP manually first; approval then triggers the SAP receipt sync. |
| 08.3 | Cancellation approval                         | 2 max             | Routing may vary by project, reason, value or refund amount.                             |
| 07.1 | Inventory shift approval                      | 1                 | Unit temporarily held while pending.                                                     |
| 10.2 | Payment Plan approval                         | 2 max             | Plan locked once approved; changes create a new version.                                 |
| 15.2 | Resale eligibility approval                   | 2 (CRM + Finance) | Lock-in period validated before submission.                                              |

**Limitation you should raise:** standard Approval Processes cannot do dynamic multi-criteria routing beyond what step criteria express, and they cap at 30 steps. BRN-08.3 says routing depends on 'Project, cancellation reason, Booking value or refund amount' - that is four dimensions. Standard approval steps can handle it with careful criteria, but if the matrix grows, Flow-based approval orchestration (or Approval Orchestration if licensed) becomes the better answer. Worth flagging before build.

## 2.5 Assignment rules, queues and round-robin

Lead Assignment Rules are standard and evaluate top-down, assigning to the first matching rule entry. They are simple and fast but cannot do load balancing or round-robin natively.

### The round-robin problem

True round-robin requires state - you must remember who got the last lead. Three patterns:

1. **Counter on a custom setting or metadata record** - increment an index, modulo the number of active users. Simple, but concurrent inserts can collide.
2. **Assignment-tracking custom object** - a row per user per project with a Last\_Assigned\_\_c timestamp and an Active\_\_c flag. Pick the eligible user with the oldest timestamp. This handles availability and load naturally.
3. **Omni-Channel** - the platform feature purpose-built for this, with real load balancing, capacity and presence. Requires the right licence but eliminates all custom routing code.

```
public with sharing class LeadRoundRobinAssigner {

    public static void assign(List<Lead> newLeads) {
        Set<Id> projectIds = new Set<Id>();
        for (Lead l : newLeads) { if (l.Project__c != null) projectIds.add(l.Project__c); }

        // Eligible, active pre-sales users per project, oldest assignment first
        Map<Id, List<Project_User_Assignment__c>> poolByProject =
            new Map<Id, List<Project_User_Assignment__c>>();

        for (Project_User_Assignment__c pua : [
            SELECT Id, Project__c, User__c, Last_Assigned__c
            FROM Project_User_Assignment__c
            WHERE Project__c IN :projectIds
            AND Is_Active__c = true
            AND Role__c = 'Pre-Sales'
            ORDER BY Last_Assigned__c ASC NULLS FIRST]) {
            if (!poolByProject.containsKey(pua.Project__c)) {
                poolByProject.put(pua.Project__c, new List<Project_User_Assignment__c>());
            }
            poolByProject.get(pua.Project__c).add(pua);
        }
    }
}
```

```

List<Project_User_Assignment__c> toUpdate = new List<Project_User_Assignment__c>();

for (Lead l : newLeads) {
    List<Project_User_Assignment__c> pool = poolByProject.get(l.Project__c);
    if (pool == null || pool.isEmpty()) {
        l.OwnerId = FallbackRouting.getFallbackQueueId(l.Project__c);    // cross-project fall
back
        continue;
    }
    Project_User_Assignment__c next = pool.remove(0);    // take the stalest
    l.OwnerId = next.User__c;
    next.Last_Assigned__c = System.now();
    pool.add(next);
    toUpdate.add(next);    // send to back of queue
}
if (!toUpdate.isEmpty()) update toUpdate;
}
}

```

**In this project:** BRN-01.6 (standard routing with cross-project fallback), BRN-01.7 (hybrid round-robin plus manual selection at the Interested stage), BRN-01.9 (handoff to Sales after first site visit), BRN-05.2 (Booking to Tower Incharge). The scope's own solution text calls for a 'project-user mapping object' with an availability flag - that is pattern 2 above. Note the recurring assumption that the availability flag is *manually maintained*; worth flagging as an operational burden the client must own.

## 3. Apex

### 3.1 Triggers and the handler pattern

One trigger per object. The trigger itself contains no logic - it delegates to a handler class. This gives you testability, control over execution order, and a single place to add recursion guards.

```
trigger BookingTrigger on Booking__c (before insert, before update,
                                     after insert, after update) {
    BookingTriggerHandler.run(trigger.operationType, trigger.new, trigger.oldMap);
}

// -----

public with sharing class BookingTriggerHandler {

    private static Boolean isRunning = false;    // recursion guard

    public static void run(System.TriggerOperation op,
                          List<Booking__c> newList,
                          Map<Id, Booking__c> oldMap) {

        if (isRunning) return;
        isRunning = true;
        try {
            switch on op {
                when BEFORE_INSERT { setDefaults(newList); }
                when BEFORE_UPDATE { stampStageChange(newList, oldMap); }
                when AFTER_INSERT { assignTowerIncharge(newList);
                                   grantCrmSharing(newList); }
                when AFTER_UPDATE { handleStageTransitions(newList, oldMap); }
            }
        } finally {
            isRunning = false;
        }
    }
}
```

#### Trigger context variables

| Variable              | Available in                   | Notes                                          |
|-----------------------|--------------------------------|------------------------------------------------|
| Trigger.new           | insert, update, undelete       | Editable in <i>before</i> contexts only.       |
| Trigger.old           | update, delete                 | Always read-only.                              |
| Trigger.newMap        | after insert, update, undelete | Not in before insert - no Ids yet.             |
| Trigger.oldMap        | update, delete                 | The workhorse for change detection.            |
| Trigger.isExecuting   | all                            | True inside trigger context.                   |
| Trigger.operationType | all                            | Enum - cleaner than chained isBefore/isInsert. |
| Trigger.size          | all                            | Number of records in this invocation.          |

#### Bulkification - the non-negotiable rule

```
// WRONG - query and DML inside a loop
for (Booking__c b : Trigger.new) {
    List<Installment__c> inst = [SELECT Id FROM Installment__c WHERE Booking__c = :b.Id];
    update inst;                // 200 queries, 200 DML
}

// RIGHT - collect, query once, act once
Set<Id> bookingIds = Trigger.newMap.keySet();

Map<Id, List<Installment__c>> byBooking = new Map<Id, List<Installment__c>>();
```

```

for (Installment__c i : [SELECT Id, Booking__c, Status__c
                        FROM Installment__c
                        WHERE Booking__c IN :bookingIds]) {
    if (!byBooking.containsKey(i.Booking__c)) {
        byBooking.put(i.Booking__c, new List<Installment__c>());
    }
    byBooking.get(i.Booking__c).add(i);
}

List<Installment__c> toUpdate = new List<Installment__c>();
for (Booking__c b : Trigger.new) {
    for (Installment__c i : byBooking.get(b.Id)) {
        i.Status__c = 'Due';
        toUpdate.add(i);
    }
}
if (!toUpdate.isEmpty()) update toUpdate;           // 1 query, 1 DML

```

### 3.2 Governor limits you will actually hit

| Limit                        | Synchronous | Asynchronous                        |
|------------------------------|-------------|-------------------------------------|
| SOQL queries                 | 100         | 200                                 |
| Records retrieved by SOQL    | 50,000      | 50,000                              |
| DML statements               | 150         | 150                                 |
| Records processed per DML    | 10,000      | 10,000                              |
| CPU time                     | 10,000 ms   | 60,000 ms                           |
| Heap size                    | 6 MB        | 12 MB                               |
| Callouts per transaction     | 100         | 100                                 |
| Total callout timeout        | 120 s       | 120 s                               |
| Future calls per transaction | 50          | 0 (cannot chain future from future) |
| Queueable jobs added         | 50          | 1 (chained)                         |

**The two that will bite you in this project:** (1) *You cannot make a callout after DML in the same synchronous transaction* - this is the single most common integration error, and it forces almost every SAP and DigiO interaction into async Apex. (2) *CPU time* - the Property Matching row (BRN-04.3) ranks units by weighted criteria; do that in a nested loop across a few thousand units and you will exceed 10 seconds of CPU.

### 3.3 Asynchronous Apex - choosing the right tool

| Type               | Signature                                                     | Use when                                                                             | Limits                                         |
|--------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------|
| <b>@future</b>     | static void, primitive params only, (callout=true) for HTTP   | Simple fire-and-forget callout. Legacy - prefer Queueable.                           | No chaining, no job Id, cannot pass sObjects.  |
| <b>Queueable</b>   | implements Queueable, Database.AllowsCallouts                 | The default choice for callouts. Accepts complex types, returns a job Id, can chain. | 50 enqueued sync; 1 chained from a Queueable.  |
| <b>Batch</b>       | implements Database.Batchable<sObject> - start/execute/finish | Processing large record volumes. Each execute() gets fresh governor limits.          | 5 concurrent jobs; default scope 200 (1-2000). |
| <b>Schedulable</b> | implements Schedulable - execute(SchedulableContext)          | Run something on a clock. Usually just kicks off a Batch.                            | 100 scheduled jobs per org.                    |

## Queueable with callout - the pattern you will write most

```
public class SapCustomerMasterSyncJob implements Queueable, Database.AllowsCallouts {

    private final Id bookingId;

    public SapCustomerMasterSyncJob(Id bookingId) { this.bookingId = bookingId; }

    public void execute(QueueableContext ctx) {

        Booking__c b = [SELECT Id, Name, Customer__c, Customer__r.Name,
                             Customer__r.PersonEmail, Customer__r.PAN__c,
                             Unit__c, Unit__r.SAP_Material_Code__c
                             FROM Booking__c WHERE Id = :bookingId LIMIT 1];

        HttpRequest req = new HttpRequest();
        req.setEndpoint('callout:SAP_Prod/api/customer-master'); // Named Credential
        req.setMethod('POST');
        req.setHeader('Content-Type', 'application/json');
        req.setTimeout(60000);
        req.setBody(SapPayloadBuilder.buildCustomerMaster(b));

        Integration_Log__c log = new Integration_Log__c(
            Interface__c = 'SAP_CUSTOMER_MASTER',
            Record_Id__c = bookingId,
            Direction__c = 'Outbound',
            Request__c = req.getBody().left(131000),
            Status__c = 'In Progress'
        );

        try {
            HttpResponse res = new Http().send(req);
            log.Response__c = res.getBody().left(131000);
            log.Status_Code__c = res.getStatusCode();

            if (res.getStatusCode() == 200 || res.getStatusCode() == 201) {
                Map<String, Object> body =
                    (Map<String, Object>) JSON.deserializeUntyped(res.getBody());
                update new Booking__c(
                    Id = bookingId,
                    SAP_Customer_Code__c = (String) body.get('customerCode'),
                    SAP_Sync_Status__c = 'Success',
                    SAP_Synced_On__c = System.now()
                );
                log.Status__c = 'Success';
            } else {
                markFailed(bookingId, 'HTTP ' + res.getStatusCode());
                log.Status__c = 'Failed';
            }
        } catch (Exception e) {
            markFailed(bookingId, e.getMessage());
            log.Status__c = 'Failed';
            log.Error__c = e.getMessage() + ' | ' + e.getStackTraceString();
        } finally {
            insert log; // ALWAYS log, success or failure
        }
    }

    private void markFailed(Id bId, String msg) {
        update new Booking__c(Id = bId, SAP_Sync_Status__c = 'Failed',
                               SAP_Sync_Error__c = msg.left(255));
    }
}
```

**In this project:** This exact shape is reused for BRN-05.5 (Customer Master), BRN-05.7 (Sales Order), BRN-06.3 (Demand trigger), BRN-06.7 (Receipt sync), BRN-08.6 (Cancellation sync), BRN-15.7 (Resale sync). BRN-18.1 explicitly asks for a *common SAP integration framework* rather than six copies of this class - that means one generic callout service, one logging object, one retry mechanism, and per-interface configuration in Custom Metadata.

## Batch Apex - overdue interest

```
public class OverdueInterestBatch
    implements Database.Batchable<sObject>, Database.Stateful {

    private Integer interestRecordsCreated = 0;

    public Database.QueryLocator start(Database.BatchableContext bc) {
        Date cutoff = Date.today().addDays(-15); // grace period from metadata
        return Database.getQueryLocator([
            SELECT Id, Booking__c, Due_Date__c, Outstanding_Amount__c,
                   Booking__r.Interest_Rate__c, Booking__r.Grace_Period_Days__c
            FROM Installment__c
            WHERE Status__c = 'Due'
                   AND Due_Date__c < :cutoff
                   AND Outstanding_Amount__c > 0
                   AND Interest_Waived__c = false
        ]);
    }

    public void execute(Database.BatchableContext bc, List<Installment__c> scope) {
        List<Installment__c> interestRows = new List<Installment__c>();
        for (Installment__c i : scope) {
            Integer overdueDays = i.Due_Date__c.daysBetween(Date.today())
                - Integer.valueOf(i.Booking__r.Grace_Period_Days__c);
            if (overdueDays <= 0) continue;

            Decimal interest = i.Outstanding_Amount__c
                * (i.Booking__r.Interest_Rate__c / 100)
                * (overdueDays / 365.0);

            // separate Interest Installment - never mutate the original
            interestRows.add(new Installment__c(
                Booking__c          = i.Booking__c,
                Type__c              = 'Interest',
                Parent_Installment__c = i.Id,
                Amount__c            = interest.setScale(2),
                Status__c            = 'Pending Demand'
            ));
        }
        if (!interestRows.isEmpty()) {
            insert interestRows;
            interestRecordsCreated += interestRows.size();
        }
    }

    public void finish(Database.BatchableContext bc) {
        // notify admin, or chain the demand-note generation batch
        System.debug('Interest rows created: ' + interestRecordsCreated);
    }
}
```

**In this project:** BRN-06.9. Note two design choices the scope explicitly requires and which you should be able to justify: (a) create a *separate* Interest Installment record rather than modifying the original - this preserves the audit trail and keeps the original demand reconcilable against SAP; (b) allow Admin waiver *before* the interest demand note is generated, which means the waiver flag must be checked in the batch query, not after.

## 3.4 Testing

75% org-wide coverage is the deployment minimum. It is not the goal. The goal is that your tests fail when the behaviour breaks.

- @isTest on the class; @testSetup for shared data creation.
- Never use SeeAllData=true unless you are testing something that genuinely cannot be created in a test (some standard pricebook scenarios).
- Test bulk: insert 200 records, not 1. This is how you catch unbulkified code.

- Test negative paths: assert that a validation rule fires, that an exception is thrown.
- Use `System.runAs()` to test sharing and permissions.
- Mock all callouts with `HttpCalloutMock` - you cannot make a real callout in a test.
- Assert. A test with no `System.assert` is coverage theatre.

```
@isTest
private class SapCustomerMasterSyncJobTest {

    private class SapMock implements HttpCalloutMock {
        private Integer code;
        SapMock(Integer code) { this.code = code; }
        public HttpResponse respond(HttpRequest req) {
            HttpResponse res = new HttpResponse();
            res.setStatusCode(code);
            res.setBody(code == 200
                ? '{"customerCode":"CUST-10021","status":"CREATED"}'
                : '{"error":"Duplicate PAN"}');
            return res;
        }
    }

    @testSetup static void setup() { TestDataFactory.createBookingWithCustomer(); }

    @isTest static void syncStoresCustomerCodeOnSuccess() {
        Booking__c b = [SELECT Id FROM Booking__c LIMIT 1];
        Test.setMock(HttpCalloutMock.class, new SapMock(200));

        Test.startTest();
        System.enqueueJob(new SapCustomerMasterSyncJob(b.Id));
        Test.stopTest(); // async completes here

        Booking__c after = [SELECT SAP_Customer_Code__c, SAP_Sync_Status__c
            FROM Booking__c WHERE Id = :b.Id];
        System.assertEquals('CUST-10021', after.SAP_Customer_Code__c);
        System.assertEquals('Success', after.SAP_Sync_Status__c);
        System.assertEquals(1, [SELECT COUNT() FROM Integration_Log__c]);
    }

    @isTest static void syncMarksFailureAndLogsOnError() {
        Booking__c b = [SELECT Id FROM Booking__c LIMIT 1];
        Test.setMock(HttpCalloutMock.class, new SapMock(500));

        Test.startTest();
        System.enqueueJob(new SapCustomerMasterSyncJob(b.Id));
        Test.stopTest();

        Booking__c after = [SELECT SAP_Sync_Status__c FROM Booking__c WHERE Id = :b.Id];
        System.assertEquals('Failed', after.SAP_Sync_Status__c);
    }
}
```

### 3.5 SOQL you must be fluent in

```
// Parent-to-child (subquery) - Booking with its Installments
SELECT Id, Name,
    (SELECT Id, Amount__c, Due_Date__c, Status__c
     FROM Installments__r
     WHERE Status__c != 'Paid'
     ORDER BY Due_Date__c)
FROM Booking__c
WHERE Status__c = 'Active'

// Child-to-parent (dot notation, up to 5 levels)
SELECT Id, Unit__r.Floor__r.Tower__r.Project__r.Name, Unit__r.Base_Price__c
FROM Booking__c

// Aggregate
```

```

SELECT Booking__c, SUM(Amount__c) totalDue, COUNT(Id) cnt
FROM Installment__c
WHERE Status__c = 'Due'
GROUP BY Booking__c
HAVING SUM(Amount__c) > 500000

// Polymorphic / TYPEOF on Task
SELECT Id, TYPEOF What WHEN Opportunity THEN StageName WHEN Account THEN Industry END
FROM Task

// FOR UPDATE - row locking, critical for unit blocking race conditions
Unit__c u = [SELECT Id, Status__c FROM Unit__c WHERE Id = :unitId FOR UPDATE];

// Security enforcement
List<Booking__c> b = [SELECT Id, Amount__c FROM Booking__c WITH SECURITY_ENFORCED];

```

**FOR UPDATE matters here.** BRN-04.12 (Conflicting Unit Selection) and BRN-04.10 (Unit Blocking) both describe a race condition: two sales executives trying to block the same unit at the same moment. A plain query-then-check-then-update has a window where both pass the check. FOR UPDATE takes a row lock so the second transaction waits. This is a genuinely good interview answer and a genuinely necessary implementation detail.

## 4. Lightning Web Components

### 4.1 When you actually need an LWC

Do not reach for LWC because it feels more developer-ish. Reach for it when the declarative UI genuinely cannot express the interaction.

| Need                                                                 | Right tool                            |
|----------------------------------------------------------------------|---------------------------------------|
| Guided linear or branching data entry                                | Screen Flow                           |
| Show/hide fields based on other fields                               | Dynamic Forms                         |
| Display related records with filters                                 | Related list or Enhanced Related List |
| Custom search with live filtering and ranking across many attributes | <b>LWC</b>                            |
| Rich interactive picker with multi-select and inline preview         | <b>LWC</b>                            |
| A visual grid or matrix (unit availability by floor)                 | <b>LWC</b>                            |
| Embedding custom UI inside a Screen Flow                             | <b>LWC</b> exposed to flow            |

**In this project:** In this scope, LWC is genuinely warranted for: Property Matching and Unit Tagging (BRN-04.3) - ranked, filterable unit picker; the Booking/Application Form public page (BRN-04.8); inventory hierarchy navigation Project > Tower > Floor > Unit (BRN-09.1); the Inventory Shift reconciliation summary screen (BRN-07.2); and the Cost Sheet preview. Most other rows should stay Screen Flow.

### 4.2 Component anatomy

```
// unitMatcher.js
import { LightningElement, api, wire, track } from 'lwc';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
import findMatchingUnits from '@salesforce/apex/UnitMatchController.findMatchingUnits';
import tagUnits          from '@salesforce/apex/UnitMatchController.tagUnitsToOpportunity';

const COLUMNS = [
  { label: 'Unit',      fieldName: 'unitName', type: 'text' },
  { label: 'Tower',     fieldName: 'tower',    type: 'text' },
  { label: 'Floor',     fieldName: 'floor',    type: 'number' },
  { label: 'Carpet',     fieldName: 'carpetArea', type: 'number' },
  { label: 'Total',     fieldName: 'totalPrice', type: 'currency',
    typeAttributes: { currencyCode: 'INR' } },
  { label: 'Score',     fieldName: 'matchScore', type: 'number' }
];

export default class UnitMatcher extends LightningElement {
  @api recordId; // injected on a record page
  columns = COLUMNS;
  units = [];
  selectedIds = [];
  isLoading = false;
  error;

  connectedCallback() { this.loadUnits(); }

  async loadUnits() {
    this.isLoading = true;
    try {
      this.units = await findMatchingUnits({ opportunityId: this.recordId });
      this.error = undefined;
    } catch (e) {
      this.error = e.body ? e.body.message : e.message;
    } finally {
      this.isLoading = false;
    }
  }
}
```

```

    }

    handleRowSelection(event) {
        this.selectedIds = event.detail.selectedRows.map(r => r.unitId);
    }

    async handleTag() {
        try {
            await tagUnits({ opportunityId: this.recordId, unitIds: this.selectedIds });
            this.dispatchEvent(new ShowToastEvent({
                title: 'Units tagged', variant: 'success'
            }));
            this.loadUnits();
        } catch (e) {
            this.dispatchEvent(new ShowToastEvent({
                title: 'Could not tag units',
                message: e.body ? e.body.message : e.message,
                variant: 'error'
            }));
        }
    }

    get hasSelection() { return this.selectedIds.length > 0; }
}

<!-- unitMatcher.html -->
<template>
    <lightning-card title="Matching Units" icon-name="custom:custom63">
        <div class="slds-p-horizontal_small">

            <template if:true={isLoading}>
                <lightning-spinner alternative-text="Loading"></lightning-spinner>
            </template>

            <template if:true={error}>
                <div class="slds-text-color_error">{error}</div>
            </template>

            <template if:true={units.length}>
                <lightning-datatable
                    key-field="unitId"
                    data={units}
                    columns={columns}
                    onrowselection={handleRowSelection}>
                </lightning-datatable>
            </template>

        </div>
        <div slot="footer">
            <lightning-button variant="brand" label="Tag Selected Units"
                disabled={hasSelection} onclick={handleTag}>
            </lightning-button>
        </div>
    </lightning-card>
</template>

```

## 4.3 @wire vs imperative Apex

|                  | @wire                                                            | Imperative                             |
|------------------|------------------------------------------------------------------|----------------------------------------|
| When it runs     | Automatically, reactively when parameters change                 | When you call it                       |
| Caching          | Cached and refreshable via refreshApex                           | Not cached unless you cache it         |
| Apex requirement | Method must be @AuraEnabled(cacheable=true) and side-effect free | Any @AuraEnabled method, including DML |

|         | @wire                                   | Imperative                            |
|---------|-----------------------------------------|---------------------------------------|
| Use for | Read-only data that should stay in sync | Actions, DML, anything user-initiated |

```
// Wire with reactive parameter
@wire(findMatchingUnits, { opportunityId: '$recordId' })
wiredUnits({ data, error }) {
    if (data) { this.units = data; this.error = undefined; }
    if (error) { this.error = error.body.message; this.units = []; }
}

// Refreshing wired data after a DML action
import { refreshApex } from '@salesforce/apex';
wiredResult;
@wire(findMatchingUnits, { opportunityId: '$recordId' })
wiredUnits(result) { this.wiredResult = result; /* ... */ }
async handleTag() { await tagUnits({...}); await refreshApex(this.wiredResult); }
```

## 4.4 Component communication

| Direction            | Mechanism                                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------------------------|
| Parent to child      | Public property with @api, or call an @api method via this.template.querySelector('c-child').methodName()          |
| Child to parent      | this.dispatchEvent(new CustomEvent('unitselected', { detail: {...} }))) and the parent listens with onunitselected |
| Unrelated components | Lightning Message Service (LMS) - publish/subscribe across the DOM, and across Aura/LWC/Visualforce                |
| Component to Flow    | @api properties marked as flow inputs/outputs in the .js-meta.xml, plus FlowAttributeChangeEvent                   |

```
<!-- Exposing an LWC to a Screen Flow: unitMatcher.js-meta.xml -->
<targets>
  <target>lightning__RecordPage</target>
  <target>lightning__FlowScreen</target>
</targets>
<targetConfigs>
  <targetConfig targets="lightning__FlowScreen">
    <property name="recordId" type="String" role="inputOnly"/>
    <property name="selectedIds" type="String" role="outputOnly"/>
  </targetConfig>
</targetConfigs>
```

## 4.5 The Apex controller behind an LWC

```
public with sharing class UnitMatchController {

    public class UnitMatchResult {
        @AuraEnabled public Id unitId { get; set; }
        @AuraEnabled public String unitName { get; set; }
        @AuraEnabled public String tower { get; set; }
        @AuraEnabled public Decimal totalPrice { get; set; }
        @AuraEnabled public Integer matchScore { get; set; }
    }

    @AuraEnabled(cacheable=true)
    public static List<UnitMatchResult> findMatchingUnits(Id opportunityId) {
        try {
            Opportunity o = [SELECT Id, Budget_Min__c, Budget_Max__c, Category__c,
                               Sub_Category__c, Preferred_Locality__c
                               FROM Opportunity WHERE Id = :opportunityId];

            List<Unit__c> candidates = [
                SELECT Id, Name, Total_Price__c, Category__c, Sub_Category__c,
```

```

        Floor__r.Tower__r.Name, Floor__r.Floor_Number__c
    FROM Unit__c
    WHERE Status__c = 'Available'
        AND Total_Price__c >= :o.Budget_Min__c
        AND Total_Price__c <= :o.Budget_Max__c
    LIMIT 200];

    return UnitScoringService.score(candidates, o); // ranking logic isolated
} catch (Exception e) {
    throw new AuraHandledException('Unable to load units: ' + e.getMessage());
}
}

@AuraEnabled
public static void tagUnitsToOpportunity(Id opportunityId, List<Id> unitIds) {
    // NOT cacheable - performs DML
    UnitTaggingService.tag(opportunityId, unitIds);
}
}

```

**Three things reviewers check:** (1) with sharing on any controller reachable from the UI, so record-level security is respected; (2) cacheable=true only on read methods; (3) exceptions wrapped in AuraHandledException so the user sees a message instead of a raw stack trace.

## 5. Integration Core Concepts

### 5.1 The integration vocabulary

| Term                       | Meaning                                                                                                                                                            |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Outbound / Inbound         | Outbound = Salesforce calls someone else. Inbound = someone calls Salesforce.                                                                                      |
| Synchronous / Asynchronous | Sync = caller waits for the answer. Async = fire and reconcile later.                                                                                              |
| Idempotency                | Processing the same message twice produces the same result as processing it once. Achieved with External IDs and upserts.                                          |
| Retry and backoff          | Re-attempting a failed call, with increasing delays, a maximum attempt count, and a dead-letter destination.                                                       |
| Payload contract           | The agreed JSON/XML structure and field meanings between the two systems.                                                                                          |
| Middleware / iPaaS         | A broker (MuleSoft, Dell Boomi, Azure Integration Services) that sits between systems. This project explicitly assumes <i>no middleware</i> - point-to-point REST. |
| Named Credential           | Salesforce's secure store for an endpoint plus its authentication, referenced as callout:Name. Keeps secrets out of code.                                          |

### 5.2 Named Credentials and External Credentials

Never put a URL, an API key, a username or a password in Apex or in Custom Metadata. Named Credentials store the endpoint; External Credentials store the authentication; a Permission Set grants a user or integration user access to the External Credential.

Setup path:

1. External Credential
  - Authentication Protocol: OAuth 2.0 / Custom / AWS Signature / JWT
  - Principal(s): Named Principal (one shared identity) or Per User
2. Named Credential
  - URL: `https://sap.bhartiyacity.internal`
  - Linked External Credential
  - Generate Authorization Header: true
  - Allow Formulas in HTTP Header/Body: as needed
3. Permission Set
  - External Credential Principal Access -> assign to the integration user

In Apex:

```
req.setEndpoint('callout:SAP_Prod/api/v1/sales-order');  
// Salesforce injects the auth header automatically - you never see the token
```

| Auth protocol                | When you will use it                                                                                            |
|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| OAuth 2.0 Client Credentials | Server-to-server with no user context. The common choice for SAP and DigiO.                                     |
| OAuth 2.0 Authorization Code | When a real user must consent - Google Drive falls here if using a user identity.                               |
| JWT Bearer                   | Certificate-based server-to-server. No refresh token to expire. Good for unattended jobs.                       |
| Custom header / API key      | Some vendors (often payment gateways) still use a static key. Store it in the External Credential, not in code. |
| Basic                        | Legacy. Acceptable only over TLS and where the vendor offers nothing better.                                    |

**In this project:** Named Credentials are specified in BRN-18.1 (SAP), BRN-18.3 (DigiO - with the important requirement of *separate* credentials for Sandbox and Production), BRN-18.4 (Easebuzz). The DigiO row also notes that Sandbox

and Production template IDs differ - so environment-specific template IDs belong in protected Custom Metadata while the secrets belong in the External Credential. Know the difference.

## 5.3 Outbound REST callout - the full anatomy

```
public class SapCalloutService {

    public class SapCalloutException extends Exception {}

    public static HttpResponse post(String interfaceName, String path, String jsonBody) {

        SAP_Interface__mdt cfg = SAP_Interface__mdt.getInstance(interfaceName);

        HttpRequest req = new HttpRequest();
        req.setEndpoint('callout:' + cfg.Named_Credential__c + path);
        req.setMethod('POST');
        req.setHeader('Content-Type', 'application/json');
        req.setHeader('Accept', 'application/json');
        req.setHeader('X-Correlation-Id', generateCorrelationId()); // traceability
        req.setTimeout(Integer.valueOf(cfg.Timeout_Ms__c));
        req.setBody(jsonBody);

        HttpResponse res = new Http().send(req);

        if (res.getStatusCode() >= 500) {
            throw new SapCalloutException('SAP server error: ' + res.getStatusCode());
        }
        if (res.getStatusCode() >= 400) {
            // 4xx is a data problem - do NOT retry, surface it
            throw new SapCalloutException('SAP rejected payload: ' + res.getBody());
        }
        return res;
    }
}
```

### The distinction that separates a junior from a senior answer

- **4xx = your fault.** Bad payload, missing mandatory field, duplicate key. Retrying sends the same bad data again. Log it, mark the record failed, surface it to a user.
- **5xx or timeout = their fault or transient.** Retry with backoff. After N attempts, dead-letter it for manual reprocessing.
- **Timeout with no response is the dangerous one.** You do not know whether SAP processed the request. This is exactly why you need idempotency on the SAP side too - agree a correlation ID or idempotency key in the contract, so a retry does not create two Sales Orders.

**Raise this in your inputs:** the scope says 'synchronous, near-real-time calls' and treats anything else as a change request. Synchronous callouts in a user transaction are fragile: a slow SAP response blocks the user, and the 120-second cumulative callout limit applies. For financial writes - Sales Order, Receipt, Cancellation - the safer pattern is async (Queueable or Platform Event) with a status field the user can see and a retry action. The scope already asks for 'support reprocessing of failed synchronization requests', which implies async whether or not it says so.

## 5.4 Inbound: Apex REST and Platform Events

### Apex REST - SAP or a vendor calls Salesforce

```
@RestResource(urlMapping='/sap/demand-note/*')
global with sharing class DemandNoteInboundApi {

    @HttpPost
    global static ResponseWrapper receiveDemandNote() {

        RestRequest req = RestContext.request;
        RestResponse res = RestContext.response;

        try {
```

```

DemandNotePayload p = (DemandNotePayload) JSON.deserialize(
    req.requestBody.toString(), DemandNotePayload.class);

// Idempotent upsert on the SAP document number
Demand_Note__c dn = new Demand_Note__c(
    SAP_Demand_Number__c = p.demandNumber,    // External Id
    Installment__c       = p.installmentsSfId,
    Amount__c            = p.amount,
    Demand_Date__c       = p.demandDate,
    Status__c            = 'Generated'
);
upsert dn SAP_Demand_Number__c;

res.statusCode = 200;
return new ResponseWrapper('SUCCESS', dn.Id, null);

} catch (Exception e) {
    IntegrationLogger.logInbound('SAP_DEMAND_NOTE',
                                req.requestBody.toString(), e);
    res.statusCode = 400;
    return new ResponseWrapper('ERROR', null, e.getMessage());
}

}

global class ResponseWrapper {
    global String status; global Id recordId; global String message;
    ResponseWrapper(String s, Id r, String m) { status=s; recordId=r; message=m; }
}

```

## Platform Events - decoupled messaging

A Platform Event is a message published to an event bus. Publishers and subscribers do not know about each other. Subscribers can be Apex triggers, Flows, LWC (via empApi), or external systems using CometD or Pub/Sub API.

```

// Publishing
Demand_Requested__e evt = new Demand_Requested__e(
    Booking_Id__c      = b.Id,
    Installment_Id__c  = i.Id,
    Amount__c          = i.Amount__c,
    Correlation_Id__c  = corrId
);
Database.SaveResult sr = EventBus.publish(evt);

// Subscribing in Apex
trigger DemandRequestedTrigger on Demand_Requested__e (after insert) {
    for (Demand_Requested__e e : Trigger.new) {
        System.enqueueJob(new SapDemandCalloutJob(e.Installment_Id__c, e.Correlation_Id__c));
    }
}

```

| Publish behaviour           | Meaning                                                                                                                 |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Publish Immediately</b>  | Event fires even if the transaction later rolls back. Use for logging and for notifying external systems of an attempt. |
| <b>Publish After Commit</b> | Event fires only if the transaction commits. Use when the event asserts that something definitely happened.             |

**In this project:** The scope proposes Platform Events as an option for BRN-06.3 (Demand Note trigger to SAP), BRN-06.7 (Receipt sync) and BRN-08.6 (Cancellation sync). The reason is sound: it decouples the user's save transaction from the callout, sidesteps the no-callout-after-DML limit cleanly, and gives you a natural replay mechanism. Know *why* you would choose Platform Events over a plain Queueable: replayability, multiple independent subscribers, and external systems being able to subscribe directly.

## 5.5 Other integration mechanisms worth knowing

| Mechanism                   | What it is                                                                                         | Relevance here                                                                                                    |
|-----------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Bulk API 2.0                | Asynchronous, high-volume record load/extract via CSV jobs.                                        | BRN-19.5 historical data migration.                                                                               |
| Composite API               | Batch several REST subrequests into one round trip, with the ability to reference earlier results. | Reduces chatter when SAP needs to write a parent and children together.                                           |
| Change Data Capture         | Publishes change events for standard/custom objects automatically.                                 | An alternative to custom Platform Events if SAP wants to consume Salesforce changes.                              |
| Salesforce Connect / OData  | Surfaces external data as External Objects, queried live, not stored.                              | Not in this scope, but the answer if the client ever asks 'why replicate SAP inventory rather than read it live'. |
| External Services           | Import an OpenAPI spec and get invocable actions usable in Flow with no Apex.                      | Viable for simple, well-documented endpoints. Worth proposing to reduce Apex volume.                              |
| Streaming API / Pub-Sub API | External systems subscribe to Salesforce events.                                                   | How SAP could listen rather than be pushed to.                                                                    |

## 6. The Named Integrations in This Project

Each of these has its own vocabulary, its own failure modes, and its own required skills. This is where most of the delivery risk in the scope sits.

### 6.1 SAP integration

#### What you are actually integrating with

SAP is not one API. Depending on the client's landscape you may be dealing with SAP S/4HANA, SAP ECC, or a real-estate-specific module (SAP RE-FX handles real estate contracts). The interface style depends on what their Basis team exposes.

| SAP interface style                                  | What it means for you                                                                                                                                                                                                  |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>OData / SAP Gateway</b>                           | The modern, preferred option. RESTful, JSON or XML, discoverable metadata. Maps cleanly to Apex HTTP callouts or even Salesforce Connect.                                                                              |
| <b>BAPI / RFC</b>                                    | Classic SAP function modules. Not directly callable over HTTP - needs SAP Gateway, SAP PI/PO, or middleware to expose them. If the client says 'we have BAPIs', that is a red flag for the 'no middleware' assumption. |
| <b>IDoc</b>                                          | SAP's batch document exchange format. Asynchronous, file or queue based. Fundamentally at odds with the 'synchronous REST' assumption in the scope.                                                                    |
| <b>SOAP web services</b>                             | Common in ECC landscapes. Salesforce can consume SOAP via WSDL2Apex, but the scope treats SOAP as a change request.                                                                                                    |
| <b>SAP PI/PO or CPI (Cloud Platform Integration)</b> | SAP's own middleware. If it exists, it is usually the right integration point - it can present a clean REST facade over BAPIs and IDocs.                                                                               |

**This is the single most important question to ask the client:** what exactly does SAP expose, and is it already REST/OData? The scope assumes 'direct point-to-point REST API integration using Named Credentials and Apex callouts' with 'no middleware efforts considered'. If SAP only exposes BAPIs or IDocs, that assumption collapses and every SAP row in the sheet is re-estimated. There are eight SAP interfaces in this scope totalling roughly 174 hours - that is a lot of effort resting on one unverified assumption.

#### The eight SAP interfaces in this scope

| BRN                      | Interface                         | Direction       | Trigger                                    |
|--------------------------|-----------------------------------|-----------------|--------------------------------------------|
| 05.5                     | Customer Master creation          | SF -> SAP -> SF | Booking approval; returns Customer Code    |
| 05.7                     | Sales Order creation              | SF -> SAP -> SF | After Customer Code; returns SO Number     |
| 05.10                    | Agreement generation              | SF -> SAP -> SF | e-Stamp receipt approved; returns document |
| 06.1                     | Construction milestone            | SAP -> SF       | Milestone marked complete in SAP           |
| 06.3                     | Demand Note creation              | SF -> SAP -> SF | User action on an installment              |
| 06.7                     | Receipt sync                      | SF -> SAP       | Finance approval of a receipt              |
| 08.6                     | Cancellation accounting           | SF -> SAP -> SF | Cancellation final approval                |
| 15.7                     | Resale completion                 | SF -> SAP -> SF | Resale process completion                  |
| 09.1 /<br>09.2 /<br>09.4 | Inventory master, status, pricing | SAP -> SF       | Master data sync                           |

#### Skills required for SAP integration

- **Apex HTTP callouts** - request construction, headers, timeouts, response parsing.

- **JSON serialisation and deserialisation** - JSON.serialize, JSON.deserialize into typed wrapper classes, deserializeUntyped for loose structures, and handling reserved words with JSONParser.
- **Named Credentials and External Credentials** - OAuth 2.0 client credentials or JWT.
- **Async Apex** - Queueable with Database.AllowsCallouts, chaining, and Batch for bulk master-data sync.
- **Platform Events** - for decoupling and replay.
- **Apex REST (inbound)** - so SAP can push milestone updates and document numbers back.
- **Error handling and logging design** - a custom Integration Log object, retry counters, dead-letter handling, admin alerting.
- **Enough SAP literacy to have the conversation** - know what a BAPI, an IDoc, a Customer Master, a Sales Order and a material code are. You do not need to write ABAP, but you must be able to sit in a design call with a SAP consultant and not be lost.
- **Postman or similar** - you will test every interface there before writing a line of Apex.

## 6.2 DigiO - document generation and e-signature

DigiO (Digio) is an Indian digital documentation platform: template-based document generation, Aadhaar eSign and other e-signature methods, document workflows, and audit trails. In this project it is the legal execution layer.

### The three modes

| Mode                   | Meaning                                                            | Where used                                                                                                                     |
|------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Generation only        | Merge Salesforce data into a template, return a PDF. No signature. | Cost Sheet (04.4), Allotment Letter (05.9), Demand Letter (06.4), Receipt Letter (06.5), NOCs (13.1), Welcome/TDS mails (05.6) |
| Signature only         | Send an existing PDF for e-signature.                              | Agreement Execution (05.11) - the agreement itself comes from SAP                                                              |
| Generation + signature | Generate and immediately route for signing.                        | Booking/Application Form (04.8), Tripartite Agreement (05.12)                                                                  |

### The integration pattern

1. Salesforce: user clicks "Generate Cost Sheet"
2. Apex: look up Digio Template ID + field mapping from Custom Metadata
3. Apex: build the merge-field map from Booking/Unit/Customer data
4. Queueable callout -> Digio /document/create (or /template/merge)
5. Digio returns a document ID -> store on the Salesforce record
- 6a. Generation only: call /document/download, store PDF as ContentVersion
- 6b. E-signature: Digio returns a signing URL; email or SMS it to the signer
7. Digio fires webhooks: sent / viewed / signed / declined / expired / completed
8. Apex REST webhook endpoint receives the event, validates the signature, updates the record status
9. On completion: download signed PDF + audit trail, store as Files
10. Log everything; allow authorised reprocessing

### What makes this hard

- **Webhook security.** Your inbound endpoint is public. You must validate that the request genuinely came from DigiO - HMAC signature verification, IP allowlisting, or a shared secret header. Never trust an unauthenticated webhook to update financial records.
- **Duplicate events.** Webhooks are delivered at-least-once. The same 'signed' event can arrive twice. Store the DigiO event ID and ignore repeats.
- **Callout limits.** Generating and downloading a document is two or three callouts. Do several at once and you approach the 100-callout and 120-second ceilings. Hence the scope's instruction to use Queueable Apex.
- **Template versioning.** Sandbox and Production have different template IDs. Store them in protected Custom Metadata per environment, not in code.

- **File storage.** Every generated PDF lands in Salesforce Files and counts against file storage. Across thousands of bookings and many document types per booking, this becomes a real storage cost - which is exactly why BRN-18.2 archives to Google Drive.

**Inconsistency worth flagging in your inputs:** the scope uses 'Digio' and 'DigiDocs' interchangeably (BRN-04.8 and BRN-05.11 say DigiDocs; everything else says Digio). Confirm these are the same vendor. Also, BRN-05.12 is titled 'Tripartite Agreement Generation' but its entire assumption and solution text describes the *Allotment Letter* - that row appears to be a copy-paste error and is estimated at 32 hours. Raise it.

## Skills required

- REST callouts, JSON, Named Credentials (as above).
- **Apex REST inbound endpoints** exposed to a public site, with signature validation.
- **ContentVersion and ContentDocumentLink** - the Salesforce Files data model.
- **Queueable chaining** - generate, then download, then link, as separate callout steps.
- **Custom Metadata-driven configuration** - template ID, merge-field map, signer roles.
- Understanding of Indian e-signature law at a basic level - Aadhaar eSign, DSC, and what makes an electronic signature legally valid under the IT Act.

## 6.3 Easebuzz - payment gateway

A payment gateway integration has one property that makes it different from every other integration in this project: **money moves**. A silent failure here is not a data quality issue, it is a customer who paid and whose booking did not progress.

### The flow

1. Salesforce generates a payment request -> Easebuzz /payment/initiateLink  
payload: amount, txnid (your reference), customer name/email/phone, success URL, failure URL, hash
2. Easebuzz returns a secure payment link
3. Link is shared with the customer (embedded in the Application Form)
4. Customer pays
5. Easebuzz calls your webhook (server-to-server) with the payment status + hash
6. Apex REST endpoint: VERIFY THE HASH, then update the record
7. Customer is redirected to the success/failure URL (browser side - never trust this alone)
8. Reconciliation: a scheduled job queries Easebuzz for any transaction still in an unknown state after N minutes

### The rules you do not break

1. **Always verify the hash.** Gateways sign their callbacks with a SHA-512 (or similar) hash built from your salt plus the transaction fields. An unverified callback is an open door to anyone who can guess your endpoint URL.
2. **Never treat the browser redirect as confirmation.** The user can close the tab, lose signal, or manipulate the URL. The server-to-server webhook is the source of truth.
3. **Always reconcile.** Webhooks get lost. A scheduled job that polls for transactions stuck in Pending and queries the gateway's status API is not optional - it is how you catch the payments that fell through the gap.
4. **Idempotency on transaction ID.** The same webhook will arrive twice. Upsert on the Easebuzz transaction ID as an External ID.
5. **Never log the full payload if it can contain card data.** Log the transaction ID and status, not raw card fields. PCI-DSS scope creeps in fast.

**In this project:** BRN-18.4, estimated at 40 hours - the highest-effort single integration row, and correctly so. Note the scope's own comment: 'Implement error handling and retry logic for failed or timed-out callbacks - Most IMPORTANT as it's related to payment'. Also note the scope limits online payment to the *token amount only*; every subsequent installment is manually reconciled by Finance. That is a significant business limitation worth confirming the client accepts.

## 6.4 CTI and the Open CTI framework

CTI (Computer Telephony Integration) connects a phone system to Salesforce so that calls can be placed from a record, incoming calls pop the matching record, and call activity is logged automatically.

### The three ways to do CTI in Salesforce

| Approach                          | What it involves                                                                                                                                                | Effort                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>Vendor AppExchange package</b> | Ozonetel, Knowlarity, Exotel, MyOperator, Servetel and similar Indian cloud telephony vendors ship managed packages. Install, configure, done.                  | Low - mostly config        |
| <b>Open CTI + custom adapter</b>  | Build a Visualforce page or LWC that uses the Open CTI JavaScript API, embedded in the softphone area of the utility bar. It talks to the vendor's browser SDK. | Medium-high - real JS work |
| <b>API-only logging</b>           | No softphone. The telephony platform pushes call records into Salesforce via REST; click-to-call opens a tel: link or triggers an outbound API call.            | Low-medium                 |

### Open CTI essentials

```
// The Open CTI JS API, loaded in a VF page or LWC in the softphone container
sforce.opencti.enableClickToDial();

sforce.opencti.onClickToDial({
  listener: function (payload) {
    // payload.number, payload.recordId, payload.objectType
    myVendorSdk.dial(payload.number);
  }
});

// Screen-pop on an inbound call
sforce.opencti.searchAndScreenPop({
  searchParams: incomingNumber,
  callType: sforce.opencti.CALL_TYPE.INBOUND,
  deferred: false,
  callback: function (result) { /* ... */ }
});

// Save the call as a Task against the matched record
sforce.opencti.saveLog({
  value: {
    entityApiName: 'Task',
    Subject: 'Inbound call',
    WhoId: leadOrContactId,
    CallDurationInSeconds: duration,
    CallDisposition: disposition,
    CallType: 'Inbound'
  },
  callback: function (response) { /* ... */ }
});
```

### Key objects and fields

- **Task** is where calls are logged. Standard fields: CallType, CallDurationInSeconds, CallDisposition, CallObject.
- WhoId points at a Lead or Contact; WhatId points at an Opportunity, Account or custom object. Getting this right is the whole of BRN-18.5.
- **Call Center** definition record - an XML definition that registers the adapter.
- **Softphone Layout** - controls what the screen-pop shows for each call type.
- **Voice / Service Cloud Voice** - Salesforce's own telephony product, if the client ever considers replacing their vendor.

**In this project:** Two separate rows: BRN-01.14 (24 hrs) scopes a full custom CTI adapter build with click-to-call, screen-pop and call logging, marked Custom. BRN-18.5 (16 hrs) says the CTI integration is *already implemented* and this is only an enhancement to linking and display, marked Config. **These two rows contradict each other.** Either the CTI exists or it does not. This is worth raising directly - there may be 24 hours of duplicate or misallocated estimate here.

## 6.5 Zapier and the lead-source integrations

Zapier is a no-code automation platform. It is used here as the integration layer for every lead source except the client's own website.

### How it works and where it breaks

| Component      | Detail                                                                                                                                                    |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trigger        | What starts the Zap - a new email in a parsed mailbox, a webhook POST, or a native app trigger (Facebook Lead Ads, Google Ads).                           |
| Action         | What it does - here, 'Find Record' in Salesforce for dedup, then 'Create Record'.                                                                         |
| Email Parser   | A Zapier tool that extracts fields from a structured email by template. Used because 99acres, MagicBricks and Housing.com have no native Zapier triggers. |
| Task limits    | Zapier bills per task. Every step in every Zap run is a task. High lead volume can get expensive fast.                                                    |
| Error handling | Zapier retains failed runs and allows replay - but only for a limited window, and only if someone is monitoring.                                          |

**The fragility to flag:** Email Parser templates break the moment a portal changes its email format, and portals change formats without notice. There is no contract, no version, no alert. Leads then silently stop arriving. Mitigations worth proposing: a scheduled Salesforce job that alerts if zero leads arrive from a given source in N hours; and asking whether the portals offer a real API or webhook (99acres and MagicBricks do have partner APIs for enterprise clients - worth checking before committing to email parsing).

### Web-to-Lead - what you should know about its limits

- 500 leads per day on most editions. Excess is queued, not lost, but is released slowly.
- No authentication - the endpoint is public and spammable. CAPTCHA is on the website side.
- Field mapping is fixed at form-generation time; adding a field means regenerating the HTML.
- No response body you can act on - it is fire and forget.
- The modern alternative is a custom form posting to an Apex REST endpoint or a Salesforce Site, which gives you validation, dedup, and a real response. Worth proposing if the client expects meaningful web traffic.

## 6.6 Google Drive integration

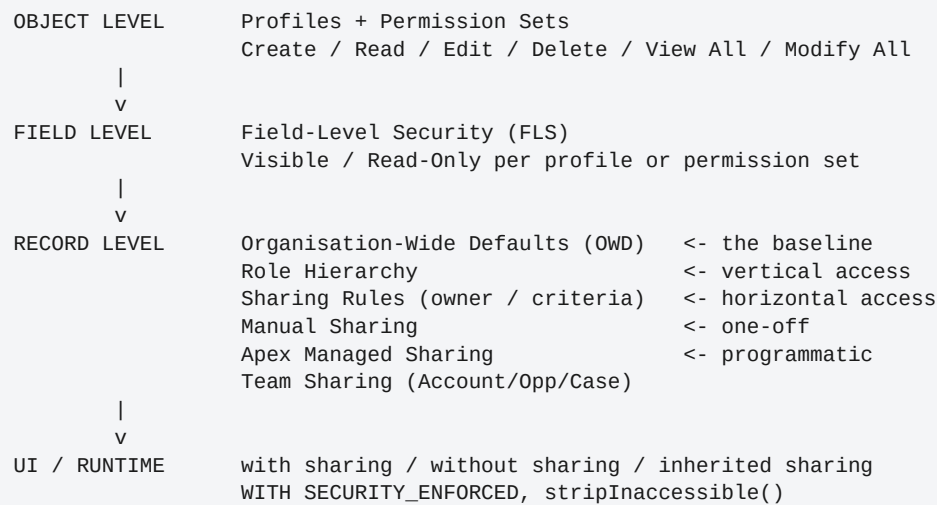
Straightforward on paper: a scheduled batch job pushes Salesforce Files into Drive folders. The complications are authentication and folder structure.

- **Auth:** OAuth 2.0. A service account with domain-wide delegation is usually right for an unattended batch job; a user-consented OAuth flow ties the archive to one person's account, which is fragile when that person leaves.
- **Reading files out of Salesforce:** ContentVersion.VersionData is a Blob. Large files will push you against the 6 MB heap limit in synchronous context and 12 MB in async - batch with a small scope size.
- **Resumable uploads:** the Drive API needs a multipart or resumable upload for anything non-trivial. This is more fiddly than a simple POST.
- **Folder creation:** you need to find-or-create the Project/Customer/Booking folder path, which means caching folder IDs rather than searching on every file.

**In this project:** BRN-18.2, 32 hours. Note carefully what the scope has *excluded*: this is a periodic archive (every 6-12 months), one-directional, with no version control and no real-time access from Salesforce. If anyone at the client believes Drive is a live working repository or a backup they can rely on for recent documents, that expectation is misaligned with what is being built. Worth stating plainly.

## 7. Security and Sharing

### 7.1 The layers, in order



### 7.2 OWD - the decision that shapes everything else

Organisation-Wide Defaults set the most restrictive baseline. You then open access upward with the role hierarchy and outward with sharing rules. You can never use sharing rules to make access *more* restrictive than the OWD.

| OWD setting          | Meaning                                                          |
|----------------------|------------------------------------------------------------------|
| Private              | Only the owner and users above them in the role hierarchy.       |
| Public Read Only     | Everyone can see, only owner and superiors can edit.             |
| Public Read/Write    | Everyone can see and edit.                                       |
| Controlled by Parent | Detail record inherits the master's access (master-detail only). |

**In this project:** BRN-17.3 requires that bank details and KYC documents are visible only to the right people. That means Booking, Applicant and any object carrying bank or KYC data should be OWD Private, with access granted deliberately. It also means FLS on the specific bank-account and PAN/Aadhaar fields, not just object-level control - a user who can see the Booking should not automatically see the account number.

### 7.3 Apex Managed Sharing - the BRN-05.2 problem

The scope states the requirement precisely: *'We need to consider the sharing of other records like customer, opportunity etc to the CRM user only after booking confirmation not before that. The Opportunity ownership remains with Sales Executive. We would have to use custom apex sharing logic for the same.'*

#### Why declarative sharing cannot do this

- Sharing rules are *criteria-based* or *owner-based*. They grant access to a group of users based on a record attribute or owner role.
- Here you need to grant access to **one specific user** (the assigned Tower Incharge) on **one specific record** at **one specific moment** (booking confirmation).
- A criteria-based rule granting the whole CRM role access to all confirmed bookings is coarser than the requirement and probably fails the data-minimisation intent.

#### How Apex Managed Sharing works

Every object with OWD below Public Read/Write gets a Share object. For custom objects it is `ObjectName__Share`; for standard objects, `OpportunityShare`, `AccountShare` and so on. You insert rows into it.

```
public with sharing class BookingSharingService {

    public static void grantCrmAccess(List<Booking__c> confirmedBookings) {

        List<Booking__Share> shares = new List<Booking__Share>();

        for (Booking__c b : confirmedBookings) {
            if (b.Tower_Incharge__c == null) continue;

            Booking__Share s = new Booking__Share();
            s.ParentId = b.Id; // the record
            s.UserOrGroupId = b.Tower_Incharge__c; // who gets access
            s.AccessLevel = 'Edit'; // Read / Edit
            s.RowCause = Schema.Booking__Share.RowCause.CRM_Handover__c;
            shares.add(s);
        }

        if (!shares.isEmpty()) {
            Database.SaveResult[] rs = Database.insert(shares, false);
            for (Integer i = 0; i < rs.size(); i++) {
                if (!rs[i].isSuccess()) {
                    System.debug('Share failed: ' + rs[i].getErrors()[0].getMessage());
                }
            }
        }
    }

    // Revoking: delete the share rows with your custom row cause
    public static void revokeCrmAccess(Set<Id> bookingIds) {
        delete [SELECT Id FROM Booking__Share
                WHERE ParentId IN :bookingIds
                AND RowCause = 'CRM_Handover__c'];
    }
}
```

### The critical detail: Apex Sharing Reasons

- An **Apex Sharing Reason** (row cause) is a custom label you define in Setup on the custom object. It tags why the share exists.
- Shares with a custom row cause survive owner changes; shares with `RowCause = 'Manual'` are **deleted automatically when the record owner changes**. That is a classic production bug.
- Custom row causes are available only on **custom objects**. For standard objects like Opportunity you are limited to 'Manual', which means you must recreate shares after ownership changes.
- Apex Managed Sharing requires the code to run in without sharing context or with Modify All permission to insert share rows.

**Design consequence to raise:** the requirement says CRM should also get access to *Customer and Opportunity*, not just Booking. Opportunity is a standard object, so you cannot use a custom row cause there - your shares will be wiped if the Opportunity owner ever changes. The cleaner alternative for Opportunity is **Opportunity Teams** (add the Tower Incharge as a team member with Edit access), which is declaratively manageable and survives correctly. Proposing that instead of Apex sharing on Opportunity is a genuinely senior observation.

## 7.4 with sharing, without sharing, inherited sharing

| Keyword      | Behaviour                                                                                |
|--------------|------------------------------------------------------------------------------------------|
| with sharing | Enforces the current user's record-level access. Use for anything reachable from the UI. |

| Keyword           | Behaviour                                                                                                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| without sharing   | Ignores record-level sharing. Use deliberately and narrowly - e.g. a service that must read a record the user cannot see in order to validate something, or to insert share rows. |
| inherited sharing | Takes the sharing mode of whatever called it. The correct choice for reusable utility classes - it is secure by default when called from a with sharing context.                  |
| No keyword        | Defaults to without sharing in most entry points. Never rely on this - always be explicit.                                                                                        |

Note that none of these keywords enforce **field-level security** or object permissions. For that:

```
// Enforce FLS + object perms at query time
List<Booking__c> b = [SELECT Id, Bank_Account_Number__c FROM Booking__c
                    WITH SECURITY_ENFORCED];

// Or strip inaccessible fields from a result set
SObjectAccessDecision d = Security.stripInaccessible(
    AccessType.READABLE, bookingList);
List<Booking__c> safe = (List<Booking__c>) d.getRecords();

// Or check explicitly
if (!Schema.sObjectType.Booking__c.fields.Bank_Account_Number__c.isAccessible()) {
    throw new SecurityException('No access');
}
```

## 7.5 Permission sets over profiles

Modern Salesforce practice: keep profiles minimal (login hours, IP ranges, default record types, page layout assignment) and grant everything else through Permission Sets and Permission Set Groups. Profiles are being de-emphasised by Salesforce over time.

**In this project:** BRN-17.2 scopes four profiles (Pre-Sales Executive, Sales Executive, Sales Coordinator, Finance) at 24 hours, and flags that a distinct CRM profile may be needed. Reading the rest of the document, the personas actually named are: Pre-Sales Executive, Receptionist, Sales Executive, Sales Coordinator, CRM Executive, Tower Incharge, CRM Manager, Finance user, Finance Head, Marketing user, Marketing Manager, Sales Manager, Leadership, CRM Admin. That is fourteen distinct personas against four scoped profiles. **This is an under-estimate worth raising explicitly.**

## 8. Documents, Files and Generation

### 8.1 The Salesforce Files data model

```
ContentDocument          the logical document (one per file, all versions)
|
+-- ContentVersion      each version; VersionData holds the Blob
|
+-- ContentDocumentLink  links the document to a record (LinkedEntityId)
                        ShareType: V (viewer) / C (collaborator) / I (inferred)
                        Visibility: AllUsers / InternalUsers / SharedUsers

// Store a generated PDF against a Booking
ContentVersion cv = new ContentVersion(
    Title           = 'Cost Sheet - ' + booking.Name,
    PathOnClient    = 'CostSheet_' + booking.Name + '.pdf',
    VersionData     = pdfBlob,
    IsMajorVersion  = true,
    Description     = 'Generated ' + System.now() + ' | Template v' + templateVersion
);
insert cv;

Id contentDocId = [SELECT ContentDocumentId FROM ContentVersion
                  WHERE Id = :cv.Id].ContentDocumentId;

insert new ContentDocumentLink(
    ContentDocumentId = contentDocId,
    LinkedEntityId    = booking.Id,
    ShareType         = 'V',
    Visibility         = 'AllUsers'
);
```

- Adding a new ContentVersion with the same ContentDocumentId creates a new *version* rather than a new document - this is how the scope's 'regenerate without overwriting previous versions' requirement is met.
- A single file can be linked to many records via multiple ContentDocumentLink rows.
- Files count against org file storage. Base64 encoding inflates a Blob by roughly 33% in transit, which matters for heap limits.
- ContentDistribution generates a public, password-optional link to a file - useful for emailing large brochures as links rather than attachments (BRN-04.5).

### 8.2 Document generation options compared

| Option                                                              | Strengths                                                                                           | Weaknesses                                                                                                                             |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Third-party platform (DigiO here)</b>                            | Business users maintain templates; e-signature built in; legal audit trail; no Apex rendering code. | Vendor dependency; per-transaction cost; every template change is a vendor-side change; API and volume limits.                         |
| <b>Visualforce renderAs='pdf'</b>                                   | Free, in-platform, full control, no external dependency.                                            | Old rendering engine, poor CSS support, no non-Latin font support without work, 10 MB output limit, developer needed for every change. |
| <b>AppExchange docgen (Conga, Nintex DocGen, S-Docs, Formstack)</b> | Mature, Word/Excel templates, business-maintainable.                                                | Licence cost per user; another vendor.                                                                                                 |
| <b>Apex + a PDF library via callout</b>                             | Full control over rendering.                                                                        | You are building and maintaining a rendering service. Rarely justified.                                                                |

**In this project:** This project standardises on DigiO, which is the right call given e-signature is needed anyway. But note BRN-09.4 hedges - 'through the document-generation tool or custom Visualforce PDF solution'. Pick one. Mixing two

document-generation approaches means two sets of templates, two failure modes and two skill requirements. Consistency is worth more than the marginal flexibility.

## 8.3 The document inventory in this project

Counting the documents this solution must produce is a good sanity check on the DigiO effort:

| Document                                    | BRN   | Signed?                     |
|---------------------------------------------|-------|-----------------------------|
| Cost Sheet                                  | 04.4  | No                          |
| Project Brochure (dispatch only)            | 04.5  | No                          |
| Booking / Application Form                  | 04.8  | Yes - customer              |
| Welcome Mail / APDS Confirmation / TDS Mail | 05.6  | No                          |
| Allotment Letter                            | 05.9  | No                          |
| Agreement (generated in SAP)                | 05.10 | Yes - customer then company |
| Tripartite Agreement                        | 05.12 | Yes                         |
| Demand Letter                               | 06.4  | No                          |
| Receipt Letter                              | 06.5  | No                          |
| Customer Ledger                             | 06.8  | No                          |
| Builder NOC / Bank NOC                      | 13.1  | No                          |

**Eleven document types, each needing a template, a merge-field map, a generation action, storage, an email dispatch, version handling and a regenerate/resend flow.** The scope estimates the shared DigiO framework at 24 hours (BRN-18.3) and then estimates each document row separately (40 + 24 + 16 + 32 + 32 + 32 + 48 ...). Check for double-counting in both directions: the framework should make each subsequent document cheap, so a 48-hour estimate for two NOC templates (BRN-13.1) after the framework exists deserves scrutiny.

## 9. Public-Facing Capture and Experience Cloud

### 9.1 The options for exposing something to a non-Salesforce user

| Option                                  | What it gives you                                                              | Licensing                                   |
|-----------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------|
| Web-to-Lead / Web-to-Case               | A generated HTML form posting straight into Salesforce. Zero auth, zero logic. | Free                                        |
| Salesforce Sites (Visualforce/Apex)     | A public page running as a guest user, with full Apex behind it.               | Free with limits                            |
| Experience Cloud - public page          | LWR or Aura site, public access, modern components, no login.                  | Free tier available; check limits           |
| Experience Cloud - authenticated portal | Logged-in customers seeing their own data.                                     | <b>Per-member or per-login licence cost</b> |
| Headless / custom front end             | Your own web app calling Salesforce APIs.                                      | API-only licence for the integration user   |

### 9.2 Guest user security - where projects get breached

The guest user is the identity anonymous visitors run as. Salesforce has tightened guest user security substantially after real-world data-exposure incidents, and you must know the current rules.

- Guest users have **Read-only OWD** enforced - you cannot give a guest user Public Read/Write on any object.
- Guest users **cannot be record owners**.
- Sharing records with guest users requires **guest user sharing rules**, which are separate and explicitly audited.
- without sharing in guest-context Apex is how data gets leaked. Be extremely deliberate.
- Never expose an object to the guest user profile that carries data beyond what the page needs. If the form needs the Unit name and price, do not give guest access to the whole Unit object with all its fields.
- The safest pattern for a form like this: the guest page holds **no** direct object access; all reads and writes go through a single without sharing Apex controller that validates a signed, expiring token first and returns only a narrow DTO.

#### The OTP-aated form pattern (BRN-04.8)

1. Sales Executive clicks "Send Booking Form" on the Opportunity
2. Apex generates a single-use token:
  - random GUID stored on a Booking\_Form\_Access\_\_c record
  - linked to the Opportunity
  - expiry timestamp (e.g. 72 hours)
  - Used\_\_c = false
3. Secure link sent by email/SMS: `https://site/booking?t=<token>`
4. Customer opens the link
  - > guest Apex validates the token: exists, not expired, not used
  - > generates a 6-digit OTP, stores its HASH (never the OTP itself)
  - > sends the OTP to the registered mobile
5. Customer enters the OTP
  - > Apex compares hashes, rate-limited (max 5 attempts, then invalidate)
  - > on success, issues a short-lived session token
6. Form loads with pre-populated data (narrow DTO only)
7. Customer completes, uploads KYC docs, pays, e-signs via DigiO
8. On submit: token marked Used, record written, verification workflow begins

**Things that must be true:** tokens are single-use and expiring; OTPs are hashed, rate-limited and time-boxed; uploaded files are validated for type and size before storage; the guest profile has the absolute minimum object access; and every access attempt is logged. The scope estimates BRN-04.8 at 32 hours - building this properly, with OTP, file upload, payment status display and e-signature initiation, is on the optimistic side of realistic.

**In this project:** BRN-11.1 notes that Experience Cloud licensing is *still pending a decision with the client*, and that ticketing must be architected now to extend to a portal later without rework. But BRN-04.8 already requires an Experience Cloud or Salesforce Site page for the booking form. Those two statements sit awkwardly together - confirm what Experience Cloud licensing actually exists, because the booking form depends on it.

## 10. Reporting, Dashboards and Data Migration

### 10.1 Report types - the thing juniors forget

You cannot report on data that no report type exposes. A custom report type defines which object is primary, which related objects can be joined (and whether the relationship is 'with' or 'with or without'), and which fields are available.

- **Primary object with related object** - only records that have children. Use for 'Bookings with Installments'.
- **Primary with or without related** - includes childless parents. Use for 'Bookings with or without Site Visits' to find bookings with none.
- Up to four object levels in a custom report type.
- Add fields via lookup to surface parent fields on the report.

#### Report formats and features

| Feature                                         | What it does                                                                                                |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Summary / Matrix report                         | Grouped rows; matrix groups rows and columns.                                                               |
| Bucket field                                    | Categorise values on the fly without a formula field.                                                       |
| Row-level formula                               | A formula evaluated per row, defined in the report.                                                         |
| Summary formula (PARENTGROUPVAL / PREVGROUPVAL) | Percentage-of-total and period-over-period comparisons.                                                     |
| Joined report                                   | Multiple report blocks side by side, different report types.                                                |
| Cross filter                                    | 'Accounts with no Opportunities' style filtering.                                                           |
| Historical Trending                             | Snapshot-based trend on up to 5 objects, limited fields.                                                    |
| Reporting Snapshots                             | Scheduled write of report results into a custom object - the real answer for point-in-time trend reporting. |

**In this project:** BRN-16.4 Leadership Dashboards asks for P&L, Marketing Spend vs ROI, Channel Mix, Sales Velocity and Investment Summary, and is the only reporting row marked Custom. That is correct - these need aggregation across Lead, Opportunity, Booking and SAP-synced financial fields, which standard report types cannot join. The honest options are: (a) a nightly Batch Apex job writing a flat summary object that dashboards read; (b) Reporting Snapshots; or (c) CRM Analytics / an external BI tool - which the scope explicitly excludes. Option (a) at 16 hours is tight.

### 10.2 Dashboards

- A dashboard component always sources from a report.
- **Dynamic dashboards** show data as the viewing user - essential when a Sales Manager and a Sales Executive should see different numbers from the same dashboard. Limited in number by edition.
- **Running user** on a static dashboard determines whose access applies - get this wrong and you either leak data or show an empty dashboard.
- Filters (up to 3) apply across components.

### 10.3 Data migration

BRN-19.5 is the single largest line item in the sheet at 50 hours, and data migration is where projects quietly fail. The technical work is not the hard part.

#### The phases

1. **Profile the source.** How many records, how clean, how many duplicates, what is genuinely required. This is where you discover the data is worse than anyone said.
2. **Agree scope and mapping.** Which objects, which fields, what transformation. Get written sign-off - the scope correctly makes this a client responsibility.
3. **Build the ETL.** Extract, cleanse, transform, dedupe, flag invalid rows.
4. **Load in dependency order.** Parents before children. Accounts, then Contacts, then Opportunities, then Bookings, then Installments.
5. **Relink relationships.** Use External IDs on every object so children can be loaded by reference rather than by looked-up Salesforce Id. This is what turns a nightmare into a routine job.
6. **Validate.** Record counts, spot checks, financial totals reconciled against the source.
7. **Rehearse.** Full dry runs in a sandbox with production-volume data, timed.

## Technical toolkit

| Tool                                             | Use                                                            |
|--------------------------------------------------|----------------------------------------------------------------|
| Bulk API 2.0                                     | Volume loads. Handles batching and retries for you.            |
| Data Loader / Data Import Wizard                 | Smaller loads, admin-driven.                                   |
| Salesforce Inspector / Dataloader.io / Workbench | Ad hoc extracts and quick loads.                               |
| Batch Apex                                       | Post-load relationship linking and derived-field population.   |
| Database.insert(list, false)                     | Partial success so one bad row does not fail 10,000 good ones. |

## Things to switch off during a load

- Validation rules that historical data cannot satisfy (a custom permission or a Bypass\_Automation\_\_c checkbox on a custom setting is the clean pattern).
- Triggers and Flows - guard them with the same bypass switch, never by deactivating them manually and forgetting to re-enable.
- Duplicate rules, if the source has known duplicates you intend to merge later.
- Email deliverability - set the org to 'System email only' so migrating 40,000 records does not email 40,000 customers. This has happened to real projects.

```
// The bypass pattern - build it before migration, keep it forever
public class AutomationBypass {
    public static Boolean isBypassed(String context) {
        Automation_Setting__c s = Automation_Setting__c.getInstance();
        if (s == null) return false;
        if (s.Bypass_All__c) return true;
        return s.get('Bypass_' + context + '__c') == true;
    }
}

// In every trigger handler
if (AutomationBypass.isBypassed('Booking')) return;
```

## 11. Skills Matrix - What to Learn, Mapped to Scope

This table is the practical answer to 'what do I need to be clear with'. Effort figures are summed from the scope sheet to show where the weight actually sits.

| Skill area                      | Specific capabilities                                                                                                   | Approx. scope hrs                        |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Data modelling</b>           | Objects, relationships, junction objects, Person Accounts, record types, External IDs, Custom Metadata design           | ~100 (BRN-02, 09, 10)                    |
| <b>Flow</b>                     | Before/after-save, scheduled, screen, scheduled paths, fault paths, bulkification, subflows, invocable Apex from Flow   | ~250 across nearly every module          |
| <b>Apex core</b>                | Trigger handler pattern, bulkification, collections, SOQL/SOSL, exception handling, wrapper classes, FOR UPDATE locking | ~300                                     |
| <b>Async Apex</b>               | Queueable with callouts, chaining, Batch, Schedulable, Database.Stateful                                                | ~150 (all integrations + BRN-06.9, 18.2) |
| <b>LWC</b>                      | Component lifecycle, wire vs imperative, datatable, events, LMS, Flow-embedded components, SLDS                         | ~120 (BRN-04.3, 04.8, 07.2, 09.1)        |
| <b>Integration - outbound</b>   | HTTP callouts, Named/External Credentials, OAuth flows, JSON ser/deser, retry and correlation IDs, integration logging  | ~200 (SAP, DigiO, Easebuzz, Drive)       |
| <b>Integration - inbound</b>    | Apex REST endpoints, webhook signature validation, idempotency, guest-user security                                     | ~80 (SAP callbacks, DigiO, Easebuzz)     |
| <b>Platform Events</b>          | Publish behaviours, event-triggered flows and triggers, replay                                                          | Used across BRN-06, 08                   |
| <b>Security and sharing</b>     | OWD, role hierarchy, sharing rules, Apex Managed Sharing, Apex Sharing Reasons, FLS enforcement, guest user hardening   | ~65 (BRN-17, 05.2)                       |
| <b>Approval processes</b>       | Multi-step, dynamic approvers, record locking, ProcessInstance auditing                                                 | ~60 across 7 approval rows               |
| <b>Files and documents</b>      | ContentVersion/ContentDocumentLink, versioning, ContentDistribution, docgen integration                                 | ~280 across 11 document types            |
| <b>Experience Cloud / Sites</b> | Public pages, guest user config, OTP patterns, file upload from guest context                                           | ~32 (BRN-04.8)                           |
| <b>Reporting</b>                | Custom report types, summary formulas, dynamic dashboards, reporting snapshots                                          | ~46 (BRN-12.3, 16)                       |
| <b>Data migration</b>           | Bulk API 2.0, External ID relinking, Batch Apex, bypass patterns, rehearsal discipline                                  | ~82 (BRN-19.5, 19.6)                     |
| <b>Telephony / CTI</b>          | Open CTI JS API, Call Center setup, softphone layouts, Task call fields                                                 | ~40 (BRN-01.14, 18.5)                    |
| <b>DevOps</b>                   | Sandbox strategy, source control, deployment (change sets, SFDX, DevOps Center), environment-specific metadata          | Implicit throughout                      |

### 11.2 Domain knowledge you also need

Platform skills alone will not get you through design workshops on this project. The vocabulary below appears throughout the scope and you are expected to already know it.

| Term                                           | Meaning                                                                                                                  |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Carpet / Built-up / Super built-up area</b> | Three ways of measuring a flat. RERA mandates carpet-area-based sale. Pricing is usually rate per sq ft on one of these. |

| Term                                        | Meaning                                                                                                                                                         |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PLC - Preferential Location Charge</b>   | A premium for a desirable unit - corner, park-facing, higher floor, extra balcony.                                                                              |
| <b>Floor Rise</b>                           | An escalating charge per floor above a base floor.                                                                                                              |
| <b>Token / Booking amount</b>               | Initial payment to reserve a unit. Here, the only amount collected online.                                                                                      |
| <b>Allotment Letter</b>                     | Formal confirmation that a specific unit is allotted to the buyer, issued after ~10% payment.                                                                   |
| <b>e-Stamp</b>                              | Electronic stamp duty payment. Legally required before an agreement can be executed in India.                                                                   |
| <b>ATS / Agreement for Sale</b>             | The binding sale agreement between builder and buyer.                                                                                                           |
| <b>APDS</b>                                 | Agreement-linked payment/demand schedule documentation referenced in the customer communications row.                                                           |
| <b>Tripartite Agreement</b>                 | Three-party agreement between builder, buyer and the buyer's lending bank. Needed for home-loan-funded purchases.                                               |
| <b>NOC - No Objection Certificate</b>       | Builder or bank confirmation that they have no objection to a transfer, name change or resale.                                                                  |
| <b>Construction-linked plan (CLP)</b>       | Payment schedule tied to construction milestones rather than dates.                                                                                             |
| <b>Assignment / Resale (pre-possession)</b> | The original buyer transferring their booking to a new buyer before possession. Usually subject to a lock-in period and an assignment charge.                   |
| <b>Snag list</b>                            | Defects found at handover inspection that must be closed before possession.                                                                                     |
| <b>Khata / Registration</b>                 | Municipal property record and legal registration of the sale deed.                                                                                              |
| <b>RERA</b>                                 | Real Estate (Regulation and Development) Act - governs project registration, broker registration, carpet-area disclosure, escrow of buyer funds, and timelines. |
| <b>Channel Partner / IPC</b>                | Broker firm routing buyers for commission. IPC = International Property Consultant (JLL, CBRE, Knight Frank and similar).                                       |

## 12. A Study Roadmap

Ordered so that each stage is usable on its own and builds toward the next. If you already have a stage, skip it - but be honest about 'I have seen it' versus 'I could build it under review'.

### Stage 1 - Foundations (you likely have most of this)

- Object and field types, relationships, junction objects.
- Validation rule formulas including ISCHANGED, PRIORVALUE, REGEX, \$Permission.
- Record types, page layouts, Dynamic Forms, compact layouts.
- OWD, role hierarchy, sharing rules, profiles vs permission sets.
- Approval processes end to end.
- **Checkpoint:** configure a Booking object with an approval process, a conditional-mandatory validation rule, and a sharing model where Finance sees all and Sales sees only their own.

### Stage 2 - Flow fluency

- All seven trigger types and when each applies.
- Before-save vs after-save and why it matters for performance.
- Loops, collection variables, and avoiding DML-in-loop.
- Scheduled paths vs scheduled-triggered flows.
- Fault paths and error logging.
- Calling Apex from Flow with @InvocableMethod.
- **Checkpoint:** build the 24-hour lead SLA breach as a scheduled flow, with a custom notification, a breach flag, and a fault path that logs failures.

### Stage 3 - Apex to production standard

- Trigger handler pattern with a recursion guard.
- Bulkification as a reflex - 200 records, always.
- Collections: Map, Set, List, and the idiom of building a Map keyed by parent Id.
- SOQL: subqueries, dot-walking, aggregates, FOR UPDATE, WITH SECURITY\_ENFORCED.
- Exception handling, custom exceptions, AuraHandledException.
- Test classes: @testSetup, bulk tests, negative tests, System.runAs, meaningful asserts.
- **Checkpoint:** write the cancellation ladder calculator as metadata-driven Apex with a test class covering three slabs, a no-match case, and a 200-record bulk insert.

### Stage 4 - Async and integration

- Queueable with Database.AllowsCallouts; chaining; when to use Batch instead.
- Named Credentials and External Credentials with OAuth 2.0 client credentials.
- Building a reusable callout service with logging, timeout and correlation IDs.
- JSON serialisation into typed wrapper classes; handling reserved words.
- HttpCalloutMock and MultiStaticResourceCalloutMock for tests.
- Apex REST inbound endpoints; webhook signature validation.
- Platform Events: publish behaviours, subscribers, replay.
- **Checkpoint:** build a mock 'SAP' endpoint in a tool like Postman Mock Server or Beeceptor, then build the full Customer Master sync against it - Queueable, Named Credential, integration log, retry action, and tests for

success, 4xx and 5xx.

## Stage 5 - LWC

- Component structure, lifecycle hooks, reactivity.
- Wire vs imperative; refreshApex.
- lightning-datatable with row selection and custom types.
- Custom events up, @api down, LMS across.
- Exposing a component to a Screen Flow.
- Error handling and toasts.
- **Checkpoint:** build the unit matcher - filter by budget and category, rank results, multi-select, tag to Opportunity, handle the empty and error states.

## Stage 6 - The hard specifics of this project

- Apex Managed Sharing with a custom Apex Sharing Reason, and its owner-change gotcha.
- Guest user hardening and the token-plus-OTP pattern.
- ContentVersion, versioning, and ContentDistribution.
- Bulk API 2.0 and External-ID-based relationship loading.
- Open CTI basics.
- Reporting snapshots and the pattern for dashboards that standard report types cannot serve.
- **Checkpoint:** implement booking handover - on confirmation, assign the Tower Incharge, grant Edit via Apex sharing with a custom row cause, add them to the Opportunity team, and prove with a test using System.runAs that access before confirmation is denied and after is granted.

## 12.2 Questions to take back to the client

Reading this scope carefully surfaces a set of open items that are worth raising as inputs. These are the ones with real estimate or delivery risk attached.

1. **What does SAP actually expose?** REST/OData, or BAPI/IDoc/SOAP? Roughly 174 hours of SAP integration rest on the assumption that it is clean REST with documented APIs and no middleware.
2. **Does the CTI integration exist or not?** BRN-01.14 (24 hrs, Custom, full adapter build) and BRN-18.5 (16 hrs, Config, 'already implemented') contradict each other.
3. **Is BRN-05.12 the Tripartite Agreement or the Allotment Letter?** The title says one thing, the entire body says the other. 32 hours is attached to it.
4. **Digio vs DigiDocs** - same vendor, or two?
5. **Experience Cloud licensing** - BRN-11.1 says the decision is pending, but BRN-04.8 requires a public site for the booking form.
6. **Four profiles against fourteen named personas** (BRN-17.2). Likely under-scoped.
7. **Can SAP hold per-unit variable charges?** Unit-level pricing with ad-hoc charges only works if SAP masters it at that granularity.
8. **Person Accounts** - irreversible, org-wide, Support-assisted. Needs explicit sign-off and early sequencing, not a buried technical assumption.
9. **Do the property portals offer real APIs?** Email parsing via Zapier is silently fragile; enterprise API access would be far more robust.
10. **Is manual Finance reconciliation genuinely acceptable** for all post-token payments (BRN-18.4), inventory-shift price differences (BRN-07.2) and resale settlements (BRN-15.5)? Three separate rows push real financial work outside the system.

11. **Field mappings deferred to discovery** across eight SAP interfaces. That is a large unbounded design effort sitting behind fixed estimates.

**Last thought.** The fastest way to get fluent in all of this is not to read about it - it is to build a scaled-down version of this exact solution in a Developer org. Project, Tower, Floor, Unit. A Lead that converts to a Person Account and an Opportunity. A unit matcher LWC. A Booking with an approval gate. A mock external endpoint you sync to. Eight objects and four integrations will teach you more than any amount of documentation, and it doubles as a portfolio piece you can walk an interviewer through.